



DEPARTMENT OF
COMPUTER SCIENCE

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

INTEGRATED MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: September 30, 2026

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

Supervisor: Miguel Goulão

Associate Professor, NOVA University Lisbon

Co-supervisor: Grischa Liebel

Associate Professor, Reykjavik University

ABSTRACT

Software and conceptual modelling are commonly included in software engineering and related computing programmes. Students do not all treat those courses as equal to their programming courses. Existing research reports that students may describe modelling as a useful instrument or as extra documentation, and that evidence is dispersed across modelling traditions and study contexts, with limited integrative explanation of how challenges arise and how they relate to strategies, support, and course conditions.

This thesis investigates how software modelling is experienced, taught, and learned in higher education, treating modelling as a socio-technical activity. It addresses three research questions: (RQ1) what challenges students and instructors perceive and how they manifest across interpreting, creating, refining, and evaluating models; (RQ2) which strategies and pedagogical or tool-related support needs they identify; and (RQ3) how tools, course design, domains, and feedback practices shape those experiences.

The study used Socio-Technical Grounded Theory. Fourteen semi-structured interviews (four lecturers, ten students) were open-coded into 277 codes and 78 memos, then compared into 14 categories and four phenomena. A second round of four student interviews, recruited from available contacts who had done software modelling, added detail inside the 14 categories and did not add a 15th. At that point the 14 categories were judged enough for the research questions of this thesis. Further details could still appear. The study does not claim that the categories were saturated.

The account shows a gated first modelling move, distinct from difficulty with unfamiliar or lookalike notation, and difficulty judging whether a model is good enough when the modelling tools the participants described often did not compile or simulate the model. A comment could still change the open work, and who could get it depended on the channel. In one case the grade came before the comments, so that delivery could not be resubmitted. Participants wanted checks and explanations rather than a tool that draws the assignment. A group can stop refining a diagram, rebuild it, or add a review so the members can explain it. Domain interest and the use of the model, a course submission or a defence, were both reported and are not ranked. Heavy workloads leave less time for worked examples and for feedback while the model can still change. These patterns

suggest possible changes to courses and tools. This study did not test whether those changes would help.

Keywords: software modelling education · conceptual modelling · grounded theory · socio-technical perspective · modelling competence

RESUMO

A modelação de software e a modelação conceptual são frequentemente incluídas em cursos de Engenharia de Software e áreas relacionadas. Nem todos os estudantes tratam essas unidades como equivalentes às de programação. A investigação existente relata que os estudantes podem descrever a modelação como um instrumento útil ou como documentação extra, e que essa evidência se encontra distribuída por diferentes tradições de modelação, com explicação integrada ainda limitada sobre o modo como os desafios emergem e se relacionam com estratégias, apoio e condições da unidade curricular.

Esta tese investiga como a modelação de software é experienciada, ensinada e aprendida no ensino superior, tratando a modelação como uma atividade socio-técnica. O estudo é guiado por três questões: (RQ1) que desafios estudantes e docentes percecionam, e como se manifestam ao interpretar, criar, refinar e avaliar modelos; (RQ2) que estratégias e necessidades de apoio (pedagógicas e de ferramentas) são identificadas; e (RQ3) de que forma ferramentas, desenho curricular, domínios e práticas de feedback moldam essas experiências.

O estudo usou Socio-Technical Grounded Theory. Catorze entrevistas semi-estruturadas (quatro docentes, dez estudantes) foram sujeitas a codificação aberta (277 códigos e 78 memorandos) e comparadas em 14 categorias e quatro fenómenos. Uma segunda vaga de quatro entrevistas a estudantes, recrutados entre contactos disponíveis que tinham feito modelação de software, acrescentou detalhe dentro das 14 categorias e não acrescentou uma décima quinta. Nesse momento, as 14 categorias foram julgadas suficientes para as questões de investigação desta tese. Podem ainda surgir mais detalhes. O estudo não afirma que as categorias estavam saturadas.

O relato mostra um primeiro passo de modelação condicionado, distinto da dificuldade com notação desconhecida ou com diagramas parecidos, e dificuldade em julgar se um modelo é suficientemente bom quando as ferramentas de modelação descritas pelos participantes muitas vezes não compilavam nem simulavam o modelo. Um comentário ainda podia alterar o trabalho em aberto, e quem conseguia obtê-lo dependia do canal. Num caso a nota chegou antes dos comentários, pelo que essa entrega não pôde ser ressubmetida. Os participantes queriam verificações e explicações, em vez de uma ferramenta que

desenhasse o trabalho. Um grupo pode deixar de refinar um diagrama, reconstruí-lo, ou acrescentar uma revisão para que os membros o consigam explicar. O interesse pelo domínio e o uso do modelo, uma entrega de cadeira ou uma defesa, foram ambos relatados e não são ordenados. Cargas de trabalho elevadas deixam menos tempo para exemplos trabalhados e para feedback enquanto o modelo ainda pode ser alterado. Estes padrões sugerem alterações possíveis aos cursos e às ferramentas. Este estudo não testou se essas alterações ajudariam.

Palavras-chave: educação em modelação de software · modelação conceptual · grounded theory · perspectiva socio-técnica · competência de modelação

CONTENTS

1	Introduction	1
1.1	Motivation	1
1.2	Problem statement and research gap	1
1.3	Aim and research questions	2
1.4	Research approach overview	3
1.5	Contributions	3
1.6	Thesis structure	4
2	Background	5
2.1	Software modelling	5
2.1.1	Purpose and role of modelling in software engineering	5
2.1.2	Conceptual modelling as a broader discipline	5
2.1.3	Modelling methods, languages, and artefacts	5
2.1.4	Modelling education	5
2.1.5	Modelling tools	6
2.1.6	Implications for the scope of this thesis	6
2.2	Bloom’s revised taxonomy	6
2.3	Grounded Theory and Socio-Technical Grounded Theory	6
2.4	Human factors	7
3	Related Work	8
3.1	Scope of the literature	8
3.2	Learning objectives	8
3.3	Reported challenges	9
3.4	Notation and tools	9
3.5	Domain, motivation, and authenticity	11
3.6	Groups and generative AI	12
3.7	How studies of this kind are done	12
3.8	What this thesis adds	12

4	Methodology	13
4.1	Research design	13
4.2	Research questions and operational focus	13
4.3	Study context and participants	14
4.4	Data collection	16
4.4.1	Interview guides and pilot	16
4.4.2	Incident-based probing	17
4.4.3	Interviews	17
4.4.4	Transcription and analytic identifiers	17
4.5	Data analysis	18
4.5.1	Coding and memoing	18
4.5.2	Category development and the findings map	19
4.5.3	The second round	19
4.5.4	Why interviewing stopped	20
4.5.5	Where Bloom was used	21
4.6	Rigour and trustworthiness	21
4.7	Ethics and data management	22
4.8	Worked example of the coding process	22
5	Findings	24
5.1	How to read this chapter	24
5.2	RQ1: Challenges across modelling activities	24
5.2.1	CAT1 — Software modeller’s block	26
5.2.2	CAT2 — Unfamiliar marks, lookalike diagrams, and a lot to absorb	27
5.2.3	CAT12 — Hard to judge good enough	28
5.2.4	CAT14 — Languages answer different questions	28
5.2.5	PH1 — Turning a written problem statement into the first diagram	30
5.3	RQ2: Strategies and support needs	30
5.3.1	CAT3 — Teacher examples and walkthroughs unlock work	31
5.3.2	CAT4 — Procedures students used to get started	31
5.3.3	CAT5 — When a comment can still be used	31
5.3.4	CAT6 — Feedback access depends on initiative and format	32
5.3.5	CAT13 — AI used to explain or compare	32
5.3.6	PH2 — A review often comes from a person	33
5.4	RQ3: Socio-technical conditions	34
5.4.1	CAT7 — Checks and guidance are wanted	34
5.4.2	CAT8 — Tool friction, switching, and cost	35
5.4.3	PH3 — Want checks and explanations, not auto-drawn homework	35
5.4.4	CAT9 — What the group did with the diagram	37
5.4.5	CAT10 — What the model is for	37
5.4.6	CAT11 — Course load and theory–practice gaps	37

5.4.7	PH4 — Shared work and what the model was for	38
5.5	Category summary	38
6	Discussion	41
6.1	Answering the research questions	41
6.1.1	RQ1 — Challenges across modelling activities	41
6.1.2	RQ2 — Strategies and support needs	43
6.1.3	RQ3 — Socio-technical conditions	44
6.2	Implications	45
6.3	Limits of the claims	46
6.4	Future work	48
7	Conclusion	50
7.1	Answers to the research questions	50
7.2	Contributions	51
7.3	Limits	51
7.4	Closing	51
Appendices		
A	Pilot interview guides	53
A.1	Instructor pilot interview guide (20–30 min)	53
A.2	Student pilot interview guide (20–30 min)	56
A.3	Mapping of interview questions to research questions	58
B	Analytic audit trail	60
B.1	How to read the cards	60
B.2	Index of reproduced memos	60
B.3	Memo cards	61
B.4	Open-code exemplars	69
Bibliography		73

LIST OF FIGURES

4.1	Analytic evidence chain used in this study. Identifiers are those stored in the vault notes; they are not timestamps. Selected memos and codes are reproduced in Appendix B.	18
4.2	Two interview rounds. The four further interviews did not add a 15th overview category. They added detail inside the existing 14.	20
5.1	Findings map. Solid groups feed a phenomenon: PH1 from CAT1, CAT3, and CAT4; PH2 from CAT12, CAT5, and CAT6; PH3 from CAT7 and CAT13; PH4 from CAT9 and CAT10. Dashed notes are not feeders. CAT2 is a different mechanism from CAT1. CAT8 contrasts with CAT7 and does not feed PH3. CAT11 limits time for demonstrations and comments around PH4. It does not feed PH4. CAT14 answers RQ1 and feeds no phenomenon. The relations used in this chapter are stated in Section 5.5. A solid arrow means those categories are part of the phenomenon. It is not a cause. A dashed line is not a cause either. The words on it say whether it is a contrast, a limit on time, or two reports side by side.	26
5.2	CAT1 variation: room-level waiting, a project that required notations S8 did not know how to use, and students who can start without copying (cases L1, S3, S5, S7, S8, S9, S10). S9, recalling that company training at a distance, still needed to see the task done once.	27
5.3	CAT1 versus CAT2. S1 could name the high-level task and still stall on the notation. That incident is CAT2. A missing path and difficulty with the notation are different mechanisms, and both can occur for the same student.	28
5.4	CAT14. An iStar model, a process model, and a class diagram answer different questions. What is left out is often what the next language is not for (S6, S10). S10 was told that afterwards, and the explanation unblocked the work. S7 is a lighter confusion about how the next language is drawn.	29
5.5	PH1 focused view. Feeding categories are CAT1, CAT3, and CAT4. CAT2 is a different mechanism and does not feed PH1.	30

5.6	CAT5 versus CAT6. When a comment arrives and who can get it are two questions. They meet when the comment arrives while the model can still be changed (S4–S10).	33
5.7	CAT13. Students asked a tool to explain a diagram or to compare options. They did not submit a generated model as the graded assignment. S5 asked for alternative class diagrams from a bullet list, compared them with the team, and for graded work rewrote the model rather than handing in the raw output. L3's wish for an explainer is not shown here.	33
5.8	PH2 focused view. CAT12 is judging a model without a run. CAT5 and CAT6 name when a comment arrives and who can get it.	34
5.9	CAT7 versus CAT8. Wanting checks and guidance is not the same as keeping the tool that offers them. L1 sits on both sides: a modelling tool is preferred when the licence allows, and licensing can still force a change. S7 dropped a checker after trying the interface. S8 never used the optional suites. S9 kept the outdated tool.	36
5.10	PH3 focused view. PH2 is the person-signal they actually get; PH3 is the check and explainer they asked for. Checking is wanted (CAT7); wanting a check did not decide what happened to the tool (CAT8); AI is used to explain or compare; graded work was rewritten rather than handed in as raw output (CAT13).	36
5.11	CAT9: refinement stopped (S4), the system was rebuilt (S7), or a review was added so the group could explain the diagrams (S10).	37
5.12	CAT10. S4 reported that an interesting domain matters. S7 described a course submission, which a teacher already reads, beside a thesis model for a supervisor and a defence. S10 treated explainability at the defence as a criterion (also CAT12). These influences are not ranked.	38
5.13	PH4 focused view. CAT9 and CAT10 feed the phenomenon. CAT11 limits time for demonstrations and comments. It does not feed PH4.	39

LIST OF TABLES

4.1	The 14 interviews (anonymised). Context is given at a general grain.	15
4.2	Prompts aimed at a Bloom level. The other prompts are not listed.	16
4.3	First-cycle codes and memos by case.	19
4.4	Cumulative overview-category count across analytic waves.	21
4.5	What the four further interviews added inside the existing categories. The marks are explained in the text. S7 and S10 incidents of switching notations are counted under CAT14, not as extra warrants for CAT2. S9's CAT6 cell is a hypothetical answer, not a recalled sharing event.	21
4.6	Illustrative coding chain from two student interviews (S1, S2). Later category IDs are those used in Chapter 5.	23
5.1	Overview of the fourteen categories. Detailed evidence follows in the sections below.	25
5.2	Where the four modelling activities are discussed. A category can be relevant to more than one activity. Listing it in a row is not a count of how often that activity occurred. CAT1, CAT2, CAT12, and CAT14 are written up under RQ1. CAT4 and CAT5 are written up under RQ2. That is a choice about where the category is discussed, not a claim that the activity happens only there. . . .	25
5.3	CAT4 procedures students used to get a first diagram started.	31
5.4	Overview categories, research question, one-line claim, and strongest memos.	40
6.1	Open questions that directed wave 2.	48
6.2	What happened after comments (wave 2). Not every comment was too late to use.	49
6.3	CAT6 access conditions. Extra comments still tend to be asked-for; the format of the channel varies.	49
A.1	Mapping of pilot interview guide questions to research questions.	59
B.1	Memos reproduced below, by overview category (same identifiers as Table 5.4).	61

INTRODUCTION

1.1 Motivation

Software systems are complex socio-technical artefacts that must evolve under technical, organisational, and human constraints. Software modelling means representing the parts of a system, or of the problem it addresses, that matter for a given purpose [25]. In practice, design models give people a shared picture they can use to try out ideas and talk about the system. Projects differ in how formal those models are and in how much they are used [17, 34]. In higher education, students meet modelling through notations such as UML, BPMN, entity–relationship models, and goal-oriented requirements models [26, 30, 31].

Despite its recognised importance, students often perceive modelling in polarised ways: as a valuable instrument for understanding and communicating about systems, or as burdensome “extra documentation” that competes with implementation time [10, 18, 27]. These perceptions are shaped by course-level conditions and by how modelling is situated within the learning workflow—for example, the tools used, the nature of modelling tasks and domains, the role of models in assessment, and how feedback is provided [18, 27]. Understanding how such conditions relate to modelling practice and competence development is important for improving modelling education and for fostering modelling skills that transfer beyond individual course contexts [31].

1.2 Problem statement and research gap

Research on modelling education has produced valuable insights, but evidence and concepts are distributed across different modelling traditions and study contexts [31]. Prior work has (i) proposed outcome-oriented frameworks and taxonomies that help articulate modelling competencies and highlight recurring gaps (e.g., limited explicit attention to evaluation and metacognitive aspects) [3, 4, 5], and (ii) reported, in method- and course-specific empirical studies, recurring difficulties such as grasping construct meaning, moving from requirements to models, and judging model adequacy, often alongside suggestions for pedagogical or tool-mediated support [26, 30].

However, there remains a need for an integrated, empirically grounded understanding of modelling education challenges and support needs that reflects both student and instructor viewpoints across educational contexts, and that connects reported difficulties to the socio-technical conditions in which modelling is taught and practiced [27, 31].

1.3 Aim and research questions

The aim of this thesis is to develop an empirically grounded account of modelling education challenges and support needs in higher education, treating modelling as a socio-technical activity that combines conceptual understanding, procedural strategies, and contextual influences.

Chakraborty and Liebel studied what students think of software modelling [10]. Liebel, Badreddin, and Heldal studied how students perceive tools and UML in model-driven courses [27]. Graßl and colleagues surveyed 90 students and 22 educators on domain choice, motivation, inclusion, and feedback [18]. This thesis asks lecturers and students about modelling challenges they can still recall, and about how they dealt with them, across several notations. Some accounts are of a named task. Others are of courses from years earlier, and the detail varies. The contribution is how those reports relate. A missing path and difficulty with the notation are different difficulties, and they are not ranked. A comment can still be used, or it can arrive after the grade. In the group episodes, what happened to the diagram went together with how the work was shared and what the model was for. The study does not measure how common those difficulties are, and it does not test whether a change would help.

The study is guided by the following research questions:

- **RQ1:** What challenges do students and instructors perceive in learning and teaching software modelling in higher education, and how do these challenges manifest across different modelling activities (e.g., interpreting, creating, refining, and evaluating models)?
- **RQ2:** What strategies do students and instructors employ to address these challenges, and what support needs (pedagogical and tool-related) do they identify?
- **RQ3:** How do socio-technical factors (e.g., tools, course design, domains, feedback practices) shape modelling learning and teaching experiences?

In line with these questions, the analysis focuses respectively on (RQ1) challenges as they appear across modelling activities, (RQ2) strategies for dealing with these challenges and articulated support needs, and (RQ3) the socio-technical conditions that shape how modelling is learned and taught.

1.4 Research approach overview

To address these questions, the thesis adopted a qualitative, theory-building approach. The study used Grounded Theory informed by a socio-technical stance that attends to interactions between social and technical elements in modelling education (Section 2.3) [21, 39]. Data were collected from four instructors and ten students (14 interviews) through semi-structured interviews. The interview questions asked participants to describe a specific episode, such as a recent assignment, a laboratory, or the start of turning a text into a model. Unlike Li et al. [26], this study did not run a tutorial and a modelling session before the interview. Some interview questions were aimed at Bloom's revised levels (Section 2.2), and the findings use those words where a challenge matched them. Codes were not sorted into Bloom levels [24]. The primary outcome is a grounded explanatory account of 14 categories and four phenomena; secondarily, the thesis derives implications for teaching practice and educational tool support, labelled as implications rather than as trialled interventions.

1.5 Contributions

This thesis contributes to software and conceptual modelling education research in four complementary ways. First, it provides an empirically grounded characterisation of the challenges that students and instructors perceive in learning and teaching modelling, including difficulties that arise across interpreting problem descriptions, constructing models, refining them after comments, and judging their quality.

Second, it offers an integrated account of the strategies participants use to deal with these challenges and the support needs they articulate. These support needs span pedagogical practices (e.g., scaffolding, feedback, assessment framing) and tool-mediated support (e.g., checking, explanation, comparison, and mechanisms for managing complexity), and are derived from concrete episodes recounted in the interviews.

Third, the thesis develops a socio-technical explanatory perspective that links modelling challenges and strategies for dealing with them to contextual factors such as tool ecosystems, course design choices, feedback practices, group work, and assignment domains. This perspective clarifies how modelling experiences are shaped by interactions between cognitive demands and the educational settings in which modelling is taught and practiced.

Finally, the thesis provides evidence-informed implications for modelling education design. These implications are formulated as recommendations for instructors and course designers and as requirements-level insights for those developing educational modelling tools. They are not claimed as validated intervention effects.

1.6 Thesis structure

The remainder of this thesis is structured as follows. Chapter 2 provides background on software and conceptual modelling, modelling education, grounded theory, Bloom-based learning outcomes, and human factors relevant to modelling learning. Chapter 3 reviews related work on modelling education frameworks, empirical studies of modelling learning challenges, tooling and model-driven engineering in education, and contextual factors such as motivation, inclusion, and authenticity. Chapter 4 presents the research design, the 14 interviews, how the data were collected and analysed, the judgment that the 14 categories were enough for the research questions of this thesis, and considerations of rigour, ethics, and limitations. Chapter 5 reports the 14 categories and four phenomena, organised by research question. Chapter 6 interprets those findings against related work, states implications, and keeps unfulfilled open questions as future work. Chapter 7 summarises the contributions and bounds of the study.

BACKGROUND

2.1 Software modelling

2.1.1 Purpose and role of modelling in software engineering

A software model shows the parts of a system, or of the problem, that matter for a purpose, such as talking about the system or trying a design [25, 17, 34]. The symbols have an agreed meaning: the same mark is read in the same way. The language is the set of allowed symbols and those meanings. A model is one description written in that language for one problem [25].

2.1.2 Conceptual modelling as a broader discipline

Conceptual modelling is the wider family, including data, process, and goal models. This thesis is about software modelling. The courses use some of those notations, such as UML, BPMN, entity-relationship models, and goal models.

2.1.3 Modelling methods, languages, and artefacts

One account of a modelling method includes the language and the steps for creating a valid model from a description. It also includes what a tool can do with that model once it exists, such as simulating it or turning it into another form [5]. The language is the allowed symbols and what they mean. What they mean is called the semantics. Students can copy the symbols and still be unable to take those steps or judge the result.

If a model is stored so a machine can read it, it can be queried, analysed, or transformed.

2.1.4 Modelling education

Modelling education, in this thesis, means the courses in which students learn software modelling: what a construct means, how to build a model from a description, and how to judge whether it is good enough. Students do not all treat those courses as equal to their programming courses. Chakraborty and Liebel found that students describe modelling either as a useful instrument or as extra documentation [10].

2.1.5 Modelling tools

A modelling tool is the program a student uses to draw a model, to check it, or to turn it into another form, such as code. In model-driven engineering, the tool treats the model as the source of that other form.

2.1.6 Implications for the scope of this thesis

This thesis is about software modelling in courses, where students and lecturers deal with what a construct means, how to build a model from a description, and how to judge whether it is good enough.

2.2 Bloom's revised taxonomy

Bloom's revised taxonomy names six levels: remember, understand, apply, analyse, evaluate, and create [24]. This thesis uses four of them for interview prompts: remembering a notation, understanding a construct, applying a procedure, and evaluating whether a model is good enough. Metacognition means noticing one's own way of working.

2.3 Grounded Theory and Socio-Technical Grounded Theory

Grounded Theory is a way of developing an explanation from qualitative material, of which interviews are one example, rather than from a hypothesis stated in advance and then tested [16, 39]. In this thesis the material is the interviews. The explanation is built from what the participants say, and from comparing those accounts with one another. The categories are not fixed in advance and then used as boxes to sort the interviews into.

Grounded Theory was first set out by Glaser and Strauss, in sociology, as a way to develop an explanation from the material rather than to test a hypothesis already in hand [16]. Later variants kept that aim and changed the procedure. Strauss and Corbin relate categories through a fixed set of headings, such as conditions, actions, and consequences [40]. Charmaz treats the explanation as an interpretation the researcher builds with the participants, not as something simply found in the data [12]. This thesis uses Socio-Technical Grounded Theory, described below, because the interviews are about people working with modelling tools, and that method was written for software engineering so that the tool stays inside the explanation.

Three steps carry the comparison. In open coding, the researcher writes a separate note for one matter in what was said, for example the notation, the tool, or the feedback, and points that note back to the place in the interview. A memo links the notes that belong to one idea. Constant comparison means that each new note is set against the notes already written, so that similar matters can be brought together and matters that only look similar can be kept apart. Chapter 4 describes how those steps were carried out in this study.

Socio-Technical Grounded Theory (STGT) was proposed for software engineering research [21]. It uses the same steps, and it was written for this field because the explanation has to cover the people, the way they work, and the tool they use. The tool is part of what is explained, not a mere setting around it. A modelling task is a case of this: the difficulty can sit in the notation, in what the lecturer did, or in the tool, and those are not separated into a human story with the tool left outside it.

This study did not use the Strauss and Corbin coding paradigm. Here, categories were related by comparing incidents and by the findings map, as Chapter 4 describes. The four further interviews were with students who were available and had done software modelling. They were not picked so that each person would answer one particular open question. The study also does not claim that interviewing had exhausted every detail of the categories.

Interviewing stopped after the four further interviews. They did not add a 15th category. At that point the 14 categories were judged enough for the research questions of this thesis. Further details could still appear. Further interviews were also difficult to arrange in the time available, as Chapter 4 states. The study does not claim that the categories were saturated. That stop is not, by itself, evidence that every relation inside the categories is settled.

2.4 Human factors

Human factors are the conditions around a task that change how a person does it. In this thesis they are the conditions around a modelling task in a course: the tool, the subject of the assignment, who reads the model, the feedback, and the load of the course. They are not the notation itself. A heavy tool, an uninteresting domain, the absence of a reader, a comment that arrives after the work can no longer change, or a heavy course load can each change what the student is able to do with the model. Chapter 3 reviews studies of those conditions. Chapters 5 and 6 describe what the participants said about them.

RELATED WORK

3.1 Scope of the literature

Research on modelling education covers data models, process models, goal models, and model-driven development. Maslov et al. map that literature and find it spread across communities that do not share one terminology or one way of evaluating teaching [31]. Findings from one family are therefore hard to compare with findings from another. The sections below follow the kinds of work in that map: frameworks for what a course might aim at, studies of where learners struggle, studies of tools, and studies of the course conditions around the task.

3.2 Learning objectives

Several frameworks use Bloom's revised taxonomy to say what a modelling course might aim at beyond copying a notation [24]. CaMeLOT combines Bloom's levels with a knowledge dimension and with areas of data-modelling content, and it gives example learning outcomes for designing a course [3]. Bogdanova and Snoeck examined assessment tasks and found that many of them ask students to understand, analyse, and create, while evaluation and metacognition receive little explicit attention [4]. Bork relates the same Bloom dimensions to the parts of a modelling method, including the language, the notation, and the procedure, and a teaching case in that paper likewise gives little explicit practice in evaluation [5]. These frameworks name a gap in written outcomes. They do not show what the lecturers in this study taught, and this thesis did not sort the interviews into Bloom levels.

Sedelmaier and Landes propose the EVELIN agenda for software engineering competence more broadly [35]. Curricula, on that agenda, should state competence targets rather than only lists of topics, and should check those targets over time. The agenda is not about modelling courses. It does treat structural conditions, such as how a programme is organised, as something a competence-oriented curriculum has to take into account.

3.3 Reported challenges

Li et al. studied beginners learning iStar. They framed tasks with Bloom's levels, analysed the accounts with grounded theory, and reported difficulty understanding constructs, applying the notation, and judging whether a model was complete [26]. From that they derived support requirements, including explanation of constructs, syntax checking, and guidance through the process, and they implemented some of those requirements in a prototype. The study included lecturers as well as beginners. It ran a tutorial and a modelling session before the interviews, and it stayed with one language. This thesis did not run that session, and it asks about several notations.

Chakraborty and Liebel interviewed students and lecturers about what modelling is for [10]. Students often did not understand what a model says, and they described modelling either as a useful instrument or as extra documentation.

Chakraborty, Troya, Burgueño, and Liebel report a separate study, in which they watched students carry out a modelling task [11]. Sixteen pairs at two universities worked for about an hour on a UML class diagram and a sequence diagram. One student drew while the other followed the problem, and then they swapped. The researchers recorded the screen, the pair's talk during the task, and the diagrams they produced, and they analysed the difficulties that showed up in that record. This thesis did not watch its participants model. The interviews are what the participants could still recount about a past incident.

Maslov sets out an agenda for BPMN education rather than the results of a completed experiment [30]. The agenda calls for a review of existing work, studies of how novices model and where they go wrong, and BPMN teaching strategies that are then tested. It places process-modelling education among the strands that still need that kind of evidence.

3.4 Notation and tools

Moody proposes principles for the design of visual notations [32]. Symbol overload means that one symbol stands for more than one construct. Symbol redundancy means that more than one symbol stands for the same construct. A diagram that simply contains many symbols is a different problem. These terms describe notations. They are not a result of the interviews in this thesis.

The UML specification separates the abstract syntax of a language from its concrete syntax [33]. The abstract syntax is the set of concepts the language allows, and the rules for combining them. The concrete syntax is how those concepts are drawn. Difficulty telling symbols apart is not the same difficulty as failing to understand the concept a symbol stands for. A diagram that follows the rules can still fail to say what the person who made it meant.

Liebel, Badreddin, and Heldal studied how students perceive tools and UML in model-driven courses [27]. What students thought of a tool depended on the course around it:

the support they were given, the place of the tool in the work, and whether the model was used to talk about requirements or had to generate code. Views of UML were more positive when models were used for requirements and analysis than when they were used mainly to generate code.

Akayama and colleagues discuss how a modelling tool is chosen for a course [2]. The choice depends on the aim of the activity, on what students already know, and on how the course is organised. The paper is a record of that discussion. It does not compare tools by how much students learn.

Hammouda and colleagues asked 44 students about assignments done on paper and assignments done in a modelling tool [19]. Students reported benefits and limits for both. Paper was used before the tool, and the assignments differed, so the medium alone does not explain the difference. The answers are perceptions, not a measure of learning.

Kienzle and colleagues, and Zschaler and colleagues, list requirements for modelling tools used in teaching, including feedback, examples, and support suited to a course [23, 44]. Both papers use one survey of 59 higher-education teachers. They are not two independent studies. A stated priority is not evidence that the feature improves learning. In that survey, support for generative AI was a relatively low priority.

Burgueño, Vallecillo, and Gogolla describe teaching UML with OCL so that a tool can execute the model and check constraints [7]. A failed check shows that a stated rule was broken. It does not show that the rule, or the model, fits the situation the student was trying to describe. They report using this approach in a course. Students who took the course said they valued it. That report is not a measured gain in learning.

A later agenda from that group sets human factors in model-driven engineering as future research: usability, the effort a tool demands, and whether the tool fits what the user is trying to do [28]. It is a list of questions for the field. It is not a second study of student perceptions.

Buchmann and colleagues argue that a model is not a drawing [6]. They call it a knowledge structure: the symbols have to follow a modelling language, so each mark stands for a concept that language allows, and the links have to be ones that language permits. A tool that only lets someone draw does not, on that account, check whether the model is a valid use of the language.

Hasker and Rowe built UMLint because a student can work for days and then wait again for a grade, and a diagram can follow the notation and still be wrong [20]. Foss, Urazova, and Lawrence treat slow marking of design diagrams as a bottleneck and offer immediate automated feedback on UML database-design questions [14]. These tools check a diagram against rules. They do not show whether a student can still change the work after a comment.

Vemuri and colleagues report a BPMN course in which students could submit optional practice exercises and receive feedback [41]. On average, those exercises drew 11 submissions from 43 students. The report does not show that the feedback improved learning. It does show that feedback can be offered and still go largely unused.

Garaccione and colleagues built a web application in which students draw BPMN diagrams, and they compared two versions of it with 200 students [15]. One version added points, progress, and penalties, and it also gave more feedback on the diagram. The other version was the same application with less feedback and without those game-like elements. Students who used the version with points and extra feedback checked the diagram more often, and on one measure the diagrams matched the intended process more closely. The results were mixed, and the authors say the tool still needs further work before ordinary classroom use. The version with the better diagrams had both the extra feedback and the points. The experiment did not include a version with only one of those, so it does not show whether the diagrams were better because of the points, because of the extra feedback, or because of the two together.

Sikkel and Daneva say textbooks rarely teach students to check that two UML diagrams of the same system agree, and they use exercises that compare a pair of diagrams [36]. Verbruggen and Snoeck watched students model one case as a class diagram and as a BPMN model [42]. Deciding which requirements belong in which diagram is, on their account, part of the quality of the model.

3.5 Domain, motivation, and authenticity

Graßl and colleagues surveyed students and educators about the domain of a modelling assignment, about motivation, and about inclusion [18]. Students preferred domains they found personally meaningful. Educators' assumptions about which domains would motivate students, such as games or technical systems, did not always match those preferences. The survey is a reason to pay attention to the domain participants describe. It does not show that a particular domain causes students to be excluded, or that a familiar domain improves learning.

Holmes et al. studied software engineering education more broadly [22]. They describe the experiences students value as real projects, real tasks, and real mentors. The work is not about modelling notations. It is about the setting of the task.

Whittle and colleagues report on a software engineering course they taught at Lancaster University [43]. They call the course a design studio: students work on a practical project, with coaching and critique, instead of lectures being the main activity. The name is borrowed from studio teaching in design schools. When students could choose how to draw, they made many informal drawings, sketches that do not follow a notation such as UML, and they almost never produced a formal model. They used those sketches when talking with each other about the design. An informal drawing can still be useful in that conversation even when it is not maintained as a formal model. The paper concerns this one course. It does not examine which assignment domain students prefer, and it does not examine inclusion.

3.6 Groups and generative AI

Silva and colleagues compared individual and group UML exercises [37]. On some measures the individual diagrams were more correct. On some measures the group diagrams were more complete. A group is not automatically the better way to model.

Cámara and colleagues tested ChatGPT on UML class diagrams with OCL and found the results unreliable next to code generation: problems of syntax, inconsistency, and scale [8]. That is a test of the system on those tasks. It is not a study of how students in a course used it.

3.7 How studies of this kind are done

Stol, Ralph, and Fitzgerald review how grounded theory has been used in software engineering and set out guidelines for doing it [39]. Hoda proposes socio-technical grounded theory for that field [21]. The explanation has to cover the people, the way they work, and the tool they use. The tool is part of what is explained, not a mere setting around it. Adolph, Hall, and Kruchten give a practical account of using grounded theory to study how software is developed [1].

This thesis used the parts named in Chapter 4: open coding, memos written during coding, and constant comparison. It did not use the Strauss and Corbin coding paradigm.

3.8 What this thesis adds

Studies cited above already include interviews with students and lecturers, comparisons of classroom tasks, and tests of tools. This thesis relates the difficulties its participants could still describe to the course conditions they described. A missing path and difficulty with the notation are reported as different difficulties and are not ranked. Comments sometimes arrived while the work could still change. In the group episodes, what happened to the diagram went together with how the work was shared and what the model was for. It does not measure how common those difficulties are, and it does not test whether a change in the course or the tool would help.

METHODOLOGY

This chapter reports how the study was designed and carried out. It converts the research questions of Chapter 1 into a qualitative procedure: who was interviewed, how data were collected and analysed, how sampling stopped, and how trustworthiness and limits are bounded. The analysis follows STGT as adapted for software engineering [21, 39].

4.1 Research design

The study used STGT [21]. The parts used were open coding, memos written during coding, and constant comparison throughout, with attention to the practices and tools in each incident. Wave 1 interviews were transcribed and coded before any further interviews. A second round was added because open questions remained. The people available for that round were contacts who had done software modelling [1, 39]. Strauss and Corbin's axial coding paradigm was not used, because categories were related through constant comparison and a findings map, not through a coding-paradigm matrix.

From Li et al. [26], this thesis took a grounded-theory approach, the revised Bloom taxonomy, and the choice to interview both lecturers and students. The grounded-theory approach here is STGT, not the Strauss and Corbin coding paradigm used in that study. Their tutorial, modelling session, and prototype tool were not used. The interviews asked about an episode the participant could still recall, and how they dealt with it, across more than one modelling method. Some participants stayed with a named task. Others remained more general.

The intended outcome was a grounded explanatory account of modelling learning and teaching challenges and associated support needs, organised as categories and cross-cutting phenomena (Chapter 5). Design-oriented recommendations are treated as implications, not as validated intervention effects.

4.2 Research questions and operational focus

The study was guided by three research questions:

- **RQ1:** What challenges do students and instructors perceive in learning and teaching software modelling in higher education, and how do these challenges manifest across modelling activities (e.g., interpreting, creating, refining, and evaluating models)?
- **RQ2:** What strategies do students and instructors employ to address these challenges, and what support needs (pedagogical and tool-related) do they identify?
- **RQ3:** How do socio-technical factors (e.g., tools, course design, domains, feedback practices) shape modelling learning and teaching experiences?

RQ1 focused on characterising challenges and their variation across modelling activities. RQ2 focused on strategies for dealing with these challenges and on articulated support needs, including the conditions under which particular forms of support were seen as helpful. RQ3 focused on linking challenges and strategies to contextual influences. The interview guides turned these three questions into things that could be asked in a conversation. Appendix [A.1](#) and Appendix [A.2](#) show those guides. Appendix [A.3](#) lists which main prompts address which research question.

4.3 Study context and participants

The study was conducted in higher-education software engineering courses and in other courses that include software modelling (for example requirements engineering, software design, business process modelling, or model-driven engineering). Two participant groups were included:

- **Students:** learners who had encountered modelling activities in at least one course and had completed at least one modelling-related assignment or project.
- **Lecturers:** people who had taught modelling-related content and had experience assessing student models or guiding modelling activities.

Fourteen interviews were completed: four lecturers (L1–L4) and ten students (S1–S10). They were collected in two rounds. Wave 1 is L1–L4 and S1–S6. Wave 2 is S7–S10. Table [4.1](#) summarises the 14 interviews in general terms.

Recruitment was through course and personal contacts. In wave 1, the students were colleagues from a software modelling course, one student whose thesis was modelling-related, and students reached through an Erasmus contact. The four lecturers were emailed directly; the addresses came through a coordinator. Those ten interviews were transcribed and coded before anyone else was asked. Open questions remained after that coding, so wave 2 was added. The four further students were contacts who had finished a computer science degree and who confirmed they had done software modelling in a course. No further lecturers were asked.

Table 4.1: The 14 interviews (anonymised). Context is given at a general grain.

ID	Role	Wave	Context (general)
L1	Lecturer	1	Portuguese HE; process, SysML, and related modelling
L2	Lecturer	1	Portuguese HE; modelling-related teaching
L3	Lecturer	1	Portuguese HE; UML-heavy undergraduate modelling and SE
L4	Lecturer	1	Portuguese HE; requirements and modelling under load
S1	Student	1	Portuguese HE; SE course, dissertation, and work sketches
S2	Student	1	Portuguese HE; SE course, dissertation, and work sketches
S3	Student	1	Portuguese HE; BPMN / process modelling
S4	Student	1	Foreign (Poland) student; UML and ER
S5	Student	1	Foreign (Poland) student; UML team project
S6	Student	1	Foreign (Germany) student; iStar, NFR, UML
S7	Student	2	Portuguese HE; systems analysis and modelling; UML / BPMN
S8	Student	2	Portuguese HE; project-required models, then master's SE
S9	Student	2	UK computer science degree, recalled weakly; later company UML training in the UK
S10	Student	2	Portuguese HE; systems analysis and modelling, several notations

Most student cases were in Portuguese higher education. Wave 2 added students at Instituto Superior Técnico (IST; S7, S10) and Instituto Superior de Engenharia de Lisboa (ISEL; S8). S9 was interviewed in Portuguese. S9 had finished a computer science degree in the United Kingdom and recalled that degree weakly. The modelling S9 could still describe is company training taken after that degree, while working at Nokia, also in the United Kingdom. S9 called it a postgraduate course. It had weekly online classes and an exam. It is not a Portuguese university course, and those incidents are not read as Portuguese university teaching. Wave 1 also included two Computer Science students at a university in Wrocław (S4, S5) and one Informatics master's student in Munich (S6). Lecturers taught modelling-related courses in Portuguese higher education.

The interviews were more useful when a participant could still describe a modelling task they remembered closely. People who could not recall what they had done, or who had last modelled a long time ago, could say less about a specific episode. Identifying course titles, project names, and personal details are omitted or generalised here.

The episodes are not all from the same point in a degree, and they are not all recent. S4 and S5 described undergraduate courses they were still taking. S6 described a master's course. S10 had finished about a year and a half before the interview. S7 had finished about three years before, and the class modelling was earlier than the dissertation. S8 placed one modelling course about four years earlier, and also described a bachelor's project before the master's. S9 finished a degree about six years before the interview, during Covid, and the modelling they could still describe was company training soon after that degree. Those company-training classes were online. The calendar dates of the interviews are not reported here.

Table 4.2: Prompts aimed at a Bloom level. The other prompts are not listed.

Guide	Question	What it asks about	Level
Student	Q5	Hardest part of a new notation: symbols, meaning, or how to start	remember, understand, and apply
Student	Q6	Steps taken to build a model from a text	apply
Student	Q7	Whether a model is good enough	evaluate
Instructor	Q5	Whether a difficulty is the meaning of a construct or how to build the model	understand and apply

4.4 Data collection

Data collection used semi-structured interviews with the two stakeholder groups. Two interview guides were prepared—one per group—with shared questions (perceptions of modelling, challenges, strategies, and support needs) but different emphases: student interviews focused on learning experiences and task-level difficulties; instructor interviews focused on teaching goals, scaffolding, assessment, and observed misconceptions. There was no time limit. Most interviews lasted 20–30 minutes; some ran longer. All fourteen interviews were conducted remotely, on Zoom or Discord. The questions asked each participant to describe a specific episode, such as a recent assignment, a laboratory, or the start of turning a text into a model, so that the interview stayed with that episode rather than with a general opinion. No live modelling exercise was run in the session, and no model artefact was present for a shared walkthrough. S1 and S2 were the first student interviews. What changed in the guide after them is described in Section 4.4.1.

Wave 2 student interviews used the same shared questions, with extra probes aimed at the open questions left by wave 1 (Appendix A).

4.4.1 Interview guides and pilot

Two semi-structured interview guides were developed: one for students and one for instructors. Some questions were aimed at Bloom’s revised levels [24]: remembering a notation, understanding a construct, applying a procedure, and judging a model. Not every question belongs to one level. Table 4.2 lists the prompts that were. The other prompts ask about context, tools, domains, or what the participant would change. Analyse and create are not given a prompt of their own.

S1 and S2 used the first version of the student guide. After those interviews the guide was changed. The shared questions stayed: an episode the participant could still recall, what was hard, and how the participant dealt with it. Later versions asked those questions more directly and added probes, including on groups, switching notations, and course load. S1 and S2 stay in the dataset because they already answered the shared questions. They are not a separate group.

4.4.2 Incident-based probing

To move beyond abstract opinions, the interviewer asked participants to recount a concrete episode: a recent modelling assignment or laboratory, a first-ten-minutes account of going from a text description to a model, or (for instructors) a recurring classroom moment when students struggled. These prompts are questions that ask for a specific episode inside the interview. They are not a second data-collection procedure.

The iStar learning study inspired the wish for situated talk: that work used a tutorial and a modelling session, then an interview about that shared experience [26]. The present study did not run such a session and did not do stimulated recall over a diagram brought to the interview. Uptake of the incident probes was uneven. Some participants stayed with a named course task; others remained more general (Section 4.6). The aim of the probes was still to surface decision points, typical errors, and tool- or feedback-related constraints that may not appear in opinion-only answers. Some questions offered a word or a possible reason. S7 was asked whether there was a compile button, and then described tests with no such button and a tool that was shown and not kept. S7 was also offered shyness, embarrassment, and not having solved the exercise as reasons nobody went to the board, and said it was a combination of those. S3 was asked whether notation was the biggest initial problem, and said yes. S10 was asked whether a change of process had been invented and whether it worked. A prompted answer is still the participant's account. It is not the same as a concern they raised before the question. Where a finding rests on that kind of question, the findings name the question.

4.4.3 Interviews

Interviews were conducted individually. The full pilot interview guides are in Appendix A.1 and Appendix A.2. Student interviews covered background and modelling exposure, perceived usefulness of modelling, learning difficulties, strategies for going from text to model, criteria for judging model quality, tool experiences, and motivational and contextual factors. Instructor interviews addressed teaching objectives, scaffolding, assessment design, common student misconceptions, tool choices and constraints, and perceived gaps in current pedagogy.

A summary mapping of guide questions to research questions is provided in Appendix A.3. Interviews included flexible probing to explore emerging themes, consistent with GT's iterative logic [1, 39]. Several wave 2 interviews (S7–S10) were conducted in Portuguese; analysis used English working transcripts, with Portuguese originals retained for verification of publication-level quotations.

4.4.4 Transcription and analytic identifiers

Interviews were transcribed, anonymised, and standardised to a speaker pair (*Interviewer* / *Participant*). Substantive turns were marked with stable identifiers (T001, T002, ...) so

that codes and memos could point back to the transcript without inventing timestamps. Identifying details were removed or generalised. The author produced the working transcripts. Identifying details were removed from the Portuguese transcripts first. An AI tool then translated those interviews (S7–S10) into English, to speed the work. The author compared each translation with the Portuguese text, side by side, so that a Portuguese expression, or an expression particular to the participants, was not lost. In the working transcripts, consent filler, repetitions, and interviewer filler were reduced where they did not affect meaning. Interviewer questions were tightened. Grammar was lightly edited for readability. The edit was not intended to change meaning, and a participant’s uncertainty was kept. The Portuguese originals are retained. Recordings were deleted after transcription, so a quotation cannot now be checked against the audio. A quotation from S7–S10 can be checked against the Portuguese original. A quotation from an interview conducted in English can be checked only against the cleaned transcript. The resulting files formed the sole first-cycle analysis corpus: 14 standardised transcripts.

4.5 Data analysis

4.5.1 Coding and memoing

Analysis was carried out by the author. Transcripts were coded in a one-note-per-code layout: each open code received a short natural-language title, one or more turn identifiers, and a short excerpt. Codes captured modelling challenges, strategies, perceptions of value, tool interactions, and contextual constraints. Constant comparison was applied throughout: newly coded incidents were compared with previous incidents and codes to refine categories and identify variation [1].

Memos were written continuously, one idea per memo, linking codes and turns. Memos recorded analytic decisions, emerging hypotheses, and links between categories [1, 39].

An open code was a separate note for one matter, such as the notation, the tool, the teaching, or the feedback. Those stayed separate notes even when the participant had described them in one episode. A memo linked the notes that belonged to its one idea, for example a notation note and a tool note, and treated that mix as one difficulty. A teaching note or a feedback note from the same participant stayed in another memo.

Linked notes kept the chain turn → code → memo → category → phenomenon (Figure 4.1).

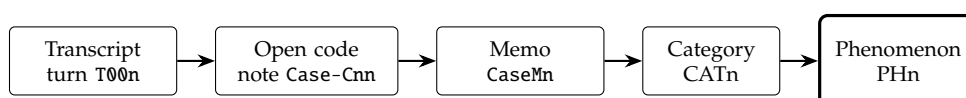


Figure 4.1: Analytic evidence chain used in this study. Identifiers are those stored in the vault notes; they are not timestamps. Selected memos and codes are reproduced in Appendix B.

Table 4.3: First-cycle codes and memos by case.

Case	Codes	Memos	Case	Codes	Memos
L1	22	6	S4	15	5
L2	24	6	S5	15	5
L3	25	7	S6	15	5
L4	35	7	S7	16	6
S1	24	6	S8	9	3
S2	30	7	S9	7	3
S3	22	6	S10	18	6
Lecturer total: 106 codes, 26 memos			Student total: 171 codes, 52 memos		

The completed first-cycle corpus comprises **277** open codes and **78** analytical memos (52 student memos; 26 lecturer memos). Per-case counts are summarised in Table 4.3.

4.5.2 Category development and the findings map

Throughout open coding and memoing, constant comparison was used to merge, split, and bound categories against the research questions. The working warrant for an overview category was multiple memos and multiple cases, unless a category was explicitly marked thin. Overloaded labels were split when memos distinguished different mechanisms (for example start-gate versus difficulty with the notation; feedback timing versus feedback access; checks and guidance versus tool friction).

The resulting overview comprises **14 categories** (CAT1–CAT14) and **4 phenomena** (PH1–PH4). Each category is assigned to one research question. In this thesis, a phenomenon is a claim that several categories support together. It is a label for that map, not a coding step from STGT. CAT2, CAT11, and CAT14 answer research questions on the overview but do not feed a phenomenon; they condition or contrast other categories.

Lecturers' accounts of their own teaching were kept separate from their interpretations of students. A support need a participant stated is reported as a finding. RQ2 asks what support they identify. The study did not test whether that support would work. A recommendation added in the discussion, beyond what participants asked for, is an implication.

4.5.3 The second round

The second round was sought after the first ten interviews had been coded, because open questions remained [1, 39]. The people who were available were not chosen to match each open question. Two analytic waves were conducted (Figure 4.2). Theoretical sampling would mean choosing each next person to develop one open question. That is not what this round was. The parts of STGT used here were open coding, memos written during coding, and constant comparison, with the tool kept in the account. That is a limited use of the approach. The 14 categories were judged enough for these research questions. They are not a finished theory.

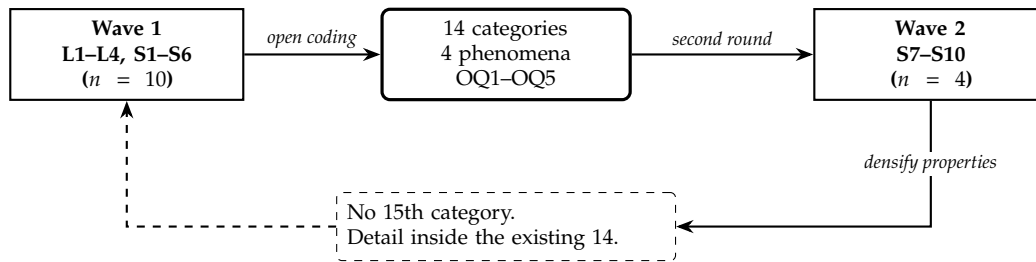


Figure 4.2: Two interview rounds. The four further interviews did not add a 15th overview category. They added detail inside the existing 14.

Wave 1. Ten interviews (L1–L4, S1–S6) were open-coded and memoed. Constant comparison produced the 14-category / 4-phenomenon overview and five open questions that were still open when the next interviews were sought (OO1–OO5; Table 6.1 in Chapter 6).

Wave 2. Four further student interviews (S7–S10) were recruited from contacts who confirmed they had done software modelling. They were not chosen one by one to match the open questions. Wave 2 added 50 first-cycle codes and 18 memos. It densified properties and bounds; it did not produce a 15th overview category that met the existing warrant. Interview density varied: S9 recalled a computer science degree in the United Kingdom only weakly, and the modelling S9 could still describe is later company training there, so some categories rest more on the other wave 2 cases (Section 4.6).

No further lecturers were asked in wave 2. Interviewing stopped after these four interviews.

4.5.4 Why interviewing stopped

Interviewing stopped for a practical reason. Further interviews were difficult to arrange, and the four further students were the contacts available. They did not add a 15th overview category. That is why interviewing stopped. It is not proof that every relation inside the 14 categories is settled. A new interview can be placed in a category that already exists and still change what that category means. Chapter 5 says which relations are actually claimed, and it names the cases. It also says where a claim is thin. Further details could still appear. The study does not claim that the categories were saturated. Table 4.4 counts the categories: 14 after the first ten interviews, and still 14 after the next four. Table 4.5 shows what those four interviews added inside the 14 categories. In that table, “new” means a new property, “boundary” means a case at the edge of the category, “known” means another example of a property already recorded, “empty” means no incident for that category, and a dash means the interview did not supply an incident strong enough to use.

Table 4.4: Cumulative overview-category count across analytic waves.

Wave	Interviews in wave	Cumulative n	Overview CATs
Wave 1 (L1–L4, S1–S6)	10	10	14
Wave 2 (S7–S10)	4	14	14

Table 4.5: What the four further interviews added inside the existing categories. The marks are explained in the text. S7 and S10 incidents of switching notations are counted under CAT14, not as extra warrants for CAT2. S9’s CAT6 cell is a hypothetical answer, not a recalled sharing event.

Category	S7	S8	S9	S10
CAT1 Starting is gated	boundary	new	known	boundary
CAT2 Unfamiliar or lookalike notation	empty	known	empty	empty
CAT3 Teacher examples	known	known	known	empty
CAT4 Self-strategies	new	known	known	known
CAT5 Feedback timing	new	known	empty	new
CAT6 Feedback access	known	new	boundary	new
CAT7 Checks and guidance	known	known	empty	empty
CAT8 Tool friction	new	known	known	empty
CAT9 Groups	new	—	—	known
CAT10 What the model is for	known	empty	empty	known
CAT11 Course load	new	new	empty	empty
CAT12 Good enough	new	empty	known	new
CAT13 AI as explainer	empty	empty	—	known
CAT14 Cross-notation switching	new	new	—	new

4.5.5 Where Bloom was used

Bloom’s revised taxonomy was used at two points [24]. When the guides were written, some questions were aimed at remembering a notation, understanding a construct, applying a procedure, and judging a model (Table 4.2). Codes and categories were not sorted into those levels. In the findings, those words are used where a challenge already matched them. That is part of reporting the analysis.

4.6 Rigour and trustworthiness

Credibility. The accounts depend on what a participant could still remember. Several students, including S7 and S9, described courses from years earlier. S9 still counts where the recall was enough to place an incident in the later company training in the United Kingdom: the start-gate (CAT1). The remark about hiding a poor drawing (CAT6) is what S9 said they would do. S9 said that situation never happened. The degree itself is recalled too weakly to carry the start-gate incident. The “—” cells for S9 under CAT9, CAT13, and CAT14 in Table 4.5 are the categories that needed a stronger incident than S9 could supply. Participants were not asked to review the categories afterwards.

Dependability and confirmability. The analysis was carried out by the author. The author had taken software modelling as a student. Some wave 1 students were colleagues from a modelling course, and the lecturer addresses came through a coordinator. Having studied the topic, and being the only person who analysed the interviews, may have introduced biases the author did not notice while conducting the interviews. One of those is to treat the course the author took as the usual way modelling is taught. Bloom's levels were used when some questions were written. Codes were not sorted into those levels. Each account was compared with the others. Accounts that went the other way were kept: S3 said notation was the bigger initial problem; S9 kept a difficult tool; S5 asked for generated alternatives and then rewrote the graded model; L4 guided students through the exercise rather than solving it first. A second researcher did not recode the transcripts, so there is no agreement score between two coders. Participants were not asked to review the categories. Those comparisons do not remove the bias. A reader can follow an excerpt to a code, a memo, and a category. Appendix B reproduces the memos named in Table 5.4 and a small set of code notes.

Transferability. The student corpus is mostly Portuguese higher education. Two Polish cases and one German case sit in wave 1. Wave 2 added IST and ISEL. For S9 it added a computer science degree in the United Kingdom, recalled weakly, and later company training in the United Kingdom, not a further Portuguese university. It added no new lecturers. Claims that come mainly from lecturers, such as heavy courses leaving little time to teach, rest on the four wave 1 lecturer interviews only. The thesis does not claim statistical generalisation. Further interviews were difficult to arrange in the time available. What these bounds mean for particular claims is taken up in Chapter 6.

4.7 Ethics and data management

Participation was voluntary and based on informed consent. Recordings were deleted after transcription. Transcripts were anonymised: course identifiers, project names, and personal data were removed or generalised. Data were stored securely and used only for the purposes of this research. When participants described modelling work or course artefacts, identifying details were removed or generalised, consistent with consent and institutional policies.

4.8 Worked example of the coding process

To illustrate the analysis procedure, this subsection shows how interview excerpts became open codes and then entered later category work. The example is drawn from two student interviews (S1, S2). It is illustrative of the chain excerpt → open code → memo → category; it is not a claim that these two cases saturate any category.

Table 4.6: Illustrative coding chain from two student interviews (S1, S2). Later category IDs are those used in Chapter 5.

Excerpt	Open codes (examples)	Later placement
S1 “Modelling is useful for defining how system components interact up to a certain point. After that, it can feel arbitrary and like extra documentation.”	value depends on context; modelling becomes extra documentation	Conditional value of modelling; not a dedicated overview CAT
S1 “Different arrows and boxes meant different things. . . it felt arbitrary when trying to do something useful with it.”	notation semantics unclear; unfamiliar marks	CAT2 (unfamiliar marks, lookalike diagrams, and a lot to absorb); contrasts with CAT1
S1 “The tool had so many options that it became completely overwhelming to know how to make a simple entity-to-box relation.”	tool feature overload; novice usability issues	CAT8 (tool friction); related to CAT2
S2 “Some concepts may be really complex and you cannot fit them in your mind. Making a small diagram can simplify them for you and for others. . . ”	diagram as cognitive offloading; communication aid	Supports why modelling is worth the cost when an audience exists (CAT10)
S2 “I had to use frameworks with which I was unfamiliar. I had to learn them before I could start drawing.”	unfamiliar method barrier; entry cost	CAT2 (unfamiliar framework), distinct from a missing path (CAT1)

Step 1: Open coding. Excerpts were segmented into meaning units and labelled with concise, action-oriented codes that stay close to the participant account (for example “symbol choices create doubt at start”, “tool options overwhelming for simple relations”).

Step 2: Memoing and comparison. Codes that recurred or contrasted were written up as short memos and compared with other cases. Relations such as “helps explain” or “contrasts with” were recorded when memos supported them; they were not forced into an axial paradigm.

Step 3: Category placement. Where patterns indicated a bounded mechanism, codes and memos were assigned to an overview category (and, where relevant, to a phenomenon). A support need a participant stated is reported as a finding, under the rule in Section 4.5.2.

This worked example is not a fixed coding template. It is included to show traceability from excerpts to codes and later categories. The corresponding open-code notes and memos for these and other cited incidents appear in Appendix B.

FINDINGS

This chapter reports the grounded account that emerged from the 14 interviews. It is organised by research question rather than as a catalogue of fourteen disconnected labels. The analysis produced **14 categories** (CAT1–CAT14) and **four phenomena** (PH1–PH4). Phenomena are used as previously defined for this thesis (Section 4.5.2). CAT2, CAT11, and CAT14 answer the research questions but do not feed a phenomenon: they contrast or condition other categories.

Quotations are short excerpts from those cleaned working transcripts, identified by case. They are not a playback of the recording. Portuguese interviews (S7–S10) are quoted from the English translation used in analysis. Claims stay bounded by memo and case identifiers; student evidence is not merged with lecturer interpretation, and design wishes are reported as support needs, not as tested interventions.

5.1 How to read this chapter

Category identifiers (CAT1–CAT14) are stable analysis labels, not a reading order. That is why the first findings section moves from CAT1 and CAT2 to CAT12 and CAT14. Each category subsection states (i) what the category is, (ii) how it varies, including wave 2 properties, (iii) at least one short quotation, and (iv) negative or boundary cases where they exist. Table 5.1 lists all fourteen categories before the detailed sections. Figure 5.1 shows how categories sit under research questions and feed phenomena. Table 5.4 at the end of the chapter repeats the map with research questions, one-line claims, and memo identifiers (reproduced in Appendix B).

5.2 RQ1: Challenges across modelling activities

RQ1 asked what challenges students and instructors perceive, and how those challenges show up when people interpret, create, refine, and evaluate models. Four categories answer this question first: getting started (CAT1), unfamiliar marks, lookalike diagrams, and a lot to absorb (CAT2), judging whether a model is good enough (CAT12), and moving

Table 5.1: Overview of the fourteen categories. Detailed evidence follows in the sections below.

ID	What it is
CAT1	Software modeller's block: the first model does not start until a demonstration or a path has been seen.
CAT2	Unfamiliar marks, lookalike diagrams, and a lot of notation to absorb.
CAT3	Teacher examples and walkthroughs are the demonstration that lets the first model start.
CAT4	Procedures students used to get a first diagram started.
CAT5	Comments sometimes arrived while the work could still change, and sometimes after the grade, when that delivery could not be resubmitted.
CAT6	Who gets detailed feedback depends on asking and on public versus private format.
CAT7	Checks and guidance are wanted. They show vocabulary or a broken language rule, not that the model fits the situation.
CAT8	Tools add cost: options, licences, switches, weak interfaces.
CAT9	A group can stop refining a diagram, rebuild it, or add a review so the members can explain it.
CAT10	S4 reported that an interesting domain matters. S7 and S10 described a course submission beside a model they had to explain at a defence. These are not ranked.
CAT11	Course load leaves less time for demonstrations and comments.
CAT12	Described tools often did not compile the model.
CAT13	AI is used to explain or compare. Graded work was not handed in as the raw output.
CAT14	Moving between languages is a change of purpose, not only of symbols.

from one kind of diagram to another (CAT14). These are different problems. Waiting for a worked example is not the same as getting lost in arrows. A class diagram can look fine as UML and still leave out the goals or the process, which that diagram may not be for. Table 5.2 shows where those activities are discussed. A category can be relevant to more than one activity. CAT4 and CAT5 are written up under RQ2. That is a choice about where the category is discussed, not a claim that the activity happens only there.

Table 5.2: Where the four modelling activities are discussed. A category can be relevant to more than one activity. Listing it in a row is not a count of how often that activity occurred. CAT1, CAT2, CAT12, and CAT14 are written up under RQ1. CAT4 and CAT5 are written up under RQ2. That is a choice about where the category is discussed, not a claim that the activity happens only there.

Activity	Where it is discussed
Interpret	CAT1, the written problem; CAT2, the marks; CAT14, what the next diagram is for
Create	CAT1, getting started; CAT4, a procedure for the first draft; CAT14, making the next diagram
Refine	CAT5, a comment while the work can still change, or after the grade; CAT4, reviewing a sketch before going over it in pen
Evaluate	CAT12, whether it is good enough; CAT5, a comment as a judgement; CAT14, whether the diagram was answering another question

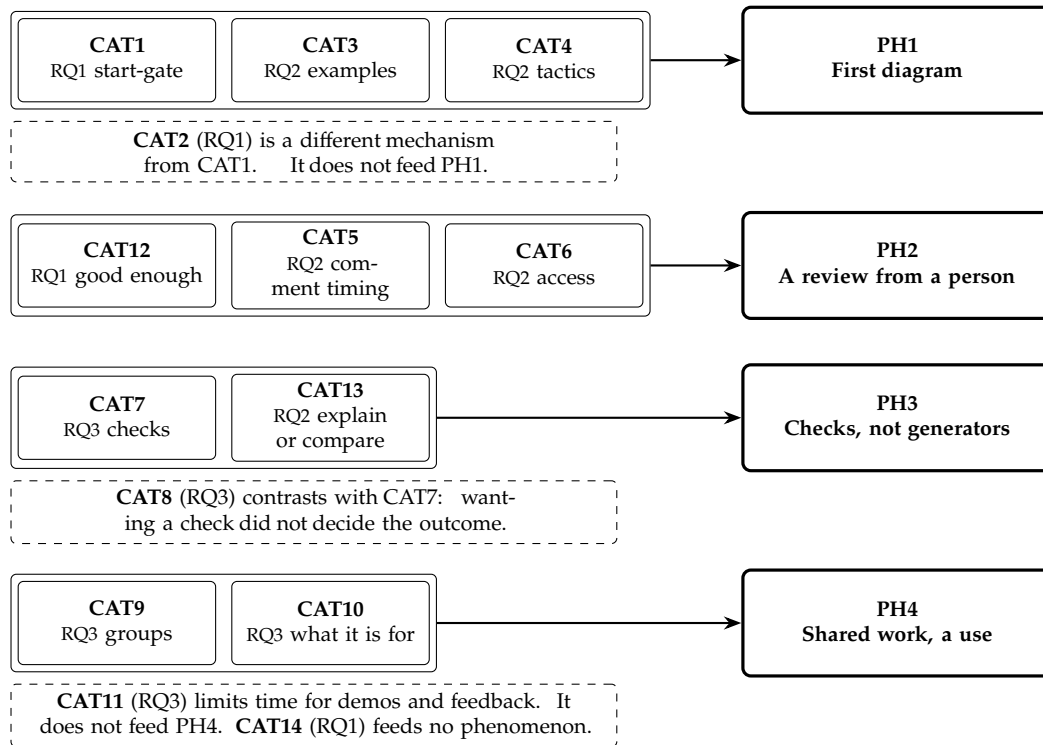


Figure 5.1: Findings map. Solid groups feed a phenomenon: PH1 from CAT1, CAT3, and CAT4; PH2 from CAT12, CAT5, and CAT6; PH3 from CAT7 and CAT13; PH4 from CAT9 and CAT10. Dashed notes are not feeders. CAT2 is a different mechanism from CAT1. CAT8 contrasts with CAT7 and does not feed PH3. CAT11 limits time for demonstrations and comments around PH4. It does not feed PH4. CAT14 answers RQ1 and feeds no phenomenon. The relations used in this chapter are stated in Section 5.5. A solid arrow means those categories are part of the phenomenon. It is not a cause. A dashed line is not a cause either. The words on it say whether it is a contrast, a limit on time, or two reports side by side.

5.2.1 CAT1 — Software modeller’s block

Getting from a written problem statement to the first real application was often gated on a demonstration, prior contact with the same notation, or some other worked path. Lecturers described a room that waits: “They wait for the teacher to start the solution—probably cultural accommodation, though I am not sure” (L1). Asked whether some students start anyway, the same lecturer added: “Some do, but they are a minority. The others wait” (L1). Students described the same gate from the other side when they had not yet seen the notation used on a live task.

Wave 2 added a different form of the same gate. In S8’s bachelor’s project, UML and requirements models were required when S8 did not know how to use those notations, even though earlier courses had already used entity-relationship and class diagrams: “I had the idea that I had to use UML, requirements engineering, and so on, because it was in the project requirements, but I had no idea how to use those notations” (S8).

The gate is not universal. S3 could start the exercises immediately in a course on

modelling systems and processes, because of earlier, even if shallow, BPMN contact: “In MSP, I remember clearly that I could start the exercises as soon as the professor showed them. I did not need to wait” (S3). S5 rarely waited for a teacher demonstration (“I rarely wait for a teacher demo”). S7 and S10 started without copying the board; wait-to-start can describe the *room* (classmates who copy) without describing every student. S9, recalling company training in the United Kingdom at a distance, likewise needed to see the task done once before starting. Figure 5.2 summarises those three forms of the gate.

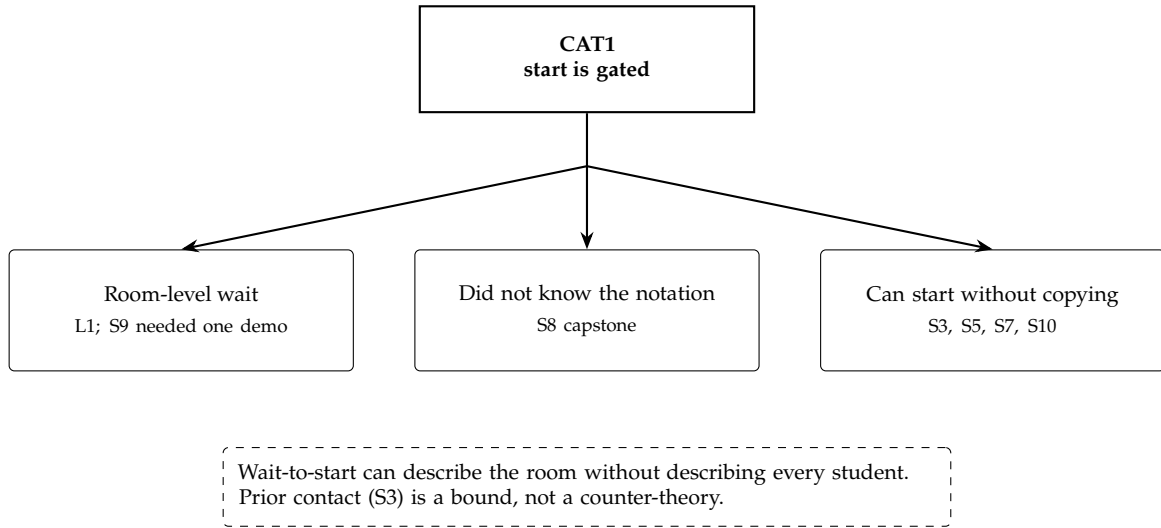


Figure 5.2: CAT1 variation: room-level waiting, a project that required notations S8 did not know how to use, and students who can start without copying (cases L1, S3, S5, S7, S8, S9, S10). S9, recalling that company training at a distance, still needed to see the task done once.

5.2.2 CAT2 — Unfamiliar marks, lookalike diagrams, and a lot to absorb

Unfamiliar symbols, arrows, and frameworks created doubt at the start *and* mid-work. This is distinct from CAT1: S1 could name the high-level task and still stall on notation. “The high-level concepts were fairly straightforward, but the symbols and notation complicated things” (S1). On a game-modelling project, “Different arrows and boxes meant different things. There was a lot of information to absorb, and it felt arbitrary when trying to do something useful with it” (S1). S2 located course difficulty in having to learn frameworks before drawing: “I had to use frameworks with which I was unfamiliar. I had to learn them before I could start drawing” (S2).

While switching notations, S8 found that “requirements diagrams looked similar to class diagrams and had similar relations as well,” and an example beside the task removed the confusion (S8). When S7 and S10 got stuck moving from one kind of diagram to another, that is discussed under CAT14 rather than as more of this notation difficulty. Figure 5.3 separates the two mechanisms. A missing path and this notation difficulty are both ways the first move gets hard, and they can happen to the same student.

Two mechanisms, not one “UML is hard” story

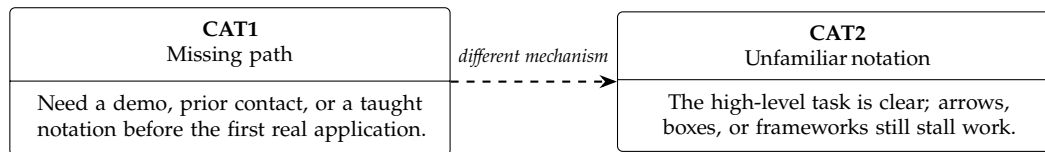


Figure 5.3: CAT1 versus CAT2. S1 could name the high-level task and still stall on the notation. That incident is CAT2. A missing path and difficulty with the notation are different mechanisms, and both can occur for the same student.

5.2.3 CAT12 — Hard to judge good enough

In this study, models and modelling tools often did not compile or simulate execution. L3 did execute student models and used that to check whether they behaved correctly. Students and lecturers described quality as a walkthrough, a justification, granularity, and readability, and more than one solution could be valid. They judged a model with checks of their own, such as explaining each element, matching the diagram to the written requirements, and readability.

S4 had no list from the instructor: “I have a kind of personal checklist, although nobody gave it to us as an official rubric” (S4). The three checks were whether every element could be explained in plain language, whether the diagram matched the written requirements, and whether it was readable enough for a supervisor. S7 treated practicals as gut feeling. On the exam the answer was not run, so S7 could not tell whether it was complete, and described the judgement as “going on faith,” after listing the requirements in ordinary language and crossing off the ones the diagram covered (S7). S10’s group also had no list from the instructor. Their questions were whether the assignment was covered, whether one model contradicted another, and whether they could explain the diagram. Explainability was the defence test: “If three people who made the model had difficulty explaining it, it was probably too complicated” (S10).

L1 accepted more than one model—“in modelling there is not only one correct way” (L1)—and still graded completeness, consistency, and readability. Those lists, and grading completeness, consistency, and readability, are how people decided the model was good enough. Nothing in the model ran and answered that question for them. A teacher’s comment was the other way people learned whether the model was good enough. When that comment arrived (CAT5), and who could get it (CAT6), are reported later, with the claim that the everyday right-or-wrong signal was a person (PH2).

5.2.4 CAT14 — Languages answer different questions

The difficulty is a change of language, not only a change of symbols. These languages, and families of languages, serve different purposes. They express different concepts, and they are drawn differently. Leaving something out of the next diagram can be the right move. A class diagram is not a goal model, so the goals are not what should be looked for inside

it. That omission is not the same thing as a missing link between the two diagrams, and it is not the same thing as a student not knowing which parts should carry. S6's hardest moment was not drawing actors but translating an iStar rationale model into UML: "our pair produced a class diagram that looked syntactically correct but lost the goals" (S6), and "nobody had explicitly explained how much of the iStar model should survive in UML" (S6). S6 also described the difference in stance: "UML felt more structural. iStar felt organisational and political: who depends on whom, which goals conflict" (S6).

S10 translated an external BPMN pool too directly into UML and was told afterwards that "the models were answering different questions" (S10). Names and requirements carried. Process structure did not become classes. S10 said that explanation unblocked the work. S10 also remembered traceability being taught, especially from requirements to system elements, but not a rigorous trace across all the models. S7's confusion at the switch was lighter, and concerned how the next language is drawn. S8's requirements diagrams and class diagrams are the similar-notation case reported under CAT2. S6 said the relation had not been explained. S10 was told after the diagram had already gone wrong. The interviews do not show that the relation had been taught before the switch. Figure 5.4 shows that limit.

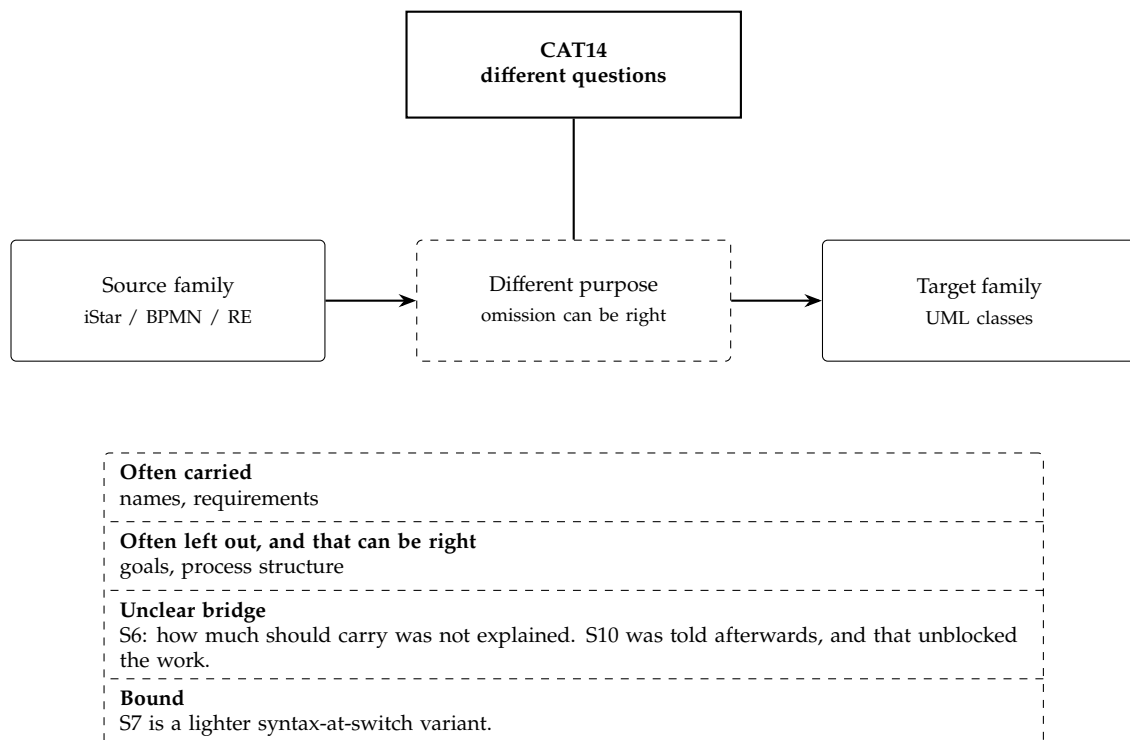


Figure 5.4: CAT14. An iStar model, a process model, and a class diagram answer different questions. What is left out is often what the next language is not for (S6, S10). S10 was told that afterwards, and the explanation unblocked the work. S7 is a lighter confusion about how the next language is drawn.

5.2.5 PH1 — Turning a written problem statement into the first diagram

Making a first model from a written problem statement is one difficulty. CAT1, CAT3, and CAT4 support that part of the account (Figure 5.5). Difficulty with the notation is another. CAT2 can delay the same first move: the marks are unfamiliar, two diagrams look alike, or there is a lot to absorb. The path itself is not what is missing. The interviews do not rank one difficulty as smaller than the other. S3, asked whether notation was the biggest initial problem, said yes: “the bigger initial problem was notation” (S3). Other students described waiting for a worked path. Those are different accounts, and both can occur for the same student.

What PH1 is *not*: a claim that every student waits, or that the symbols do not matter. S3, S5, S7, and S10 could start; the room can still wait. Teacher examples open the move from the problem statement to a first diagram (CAT3). CAT4 reports procedures some students used while making that move.

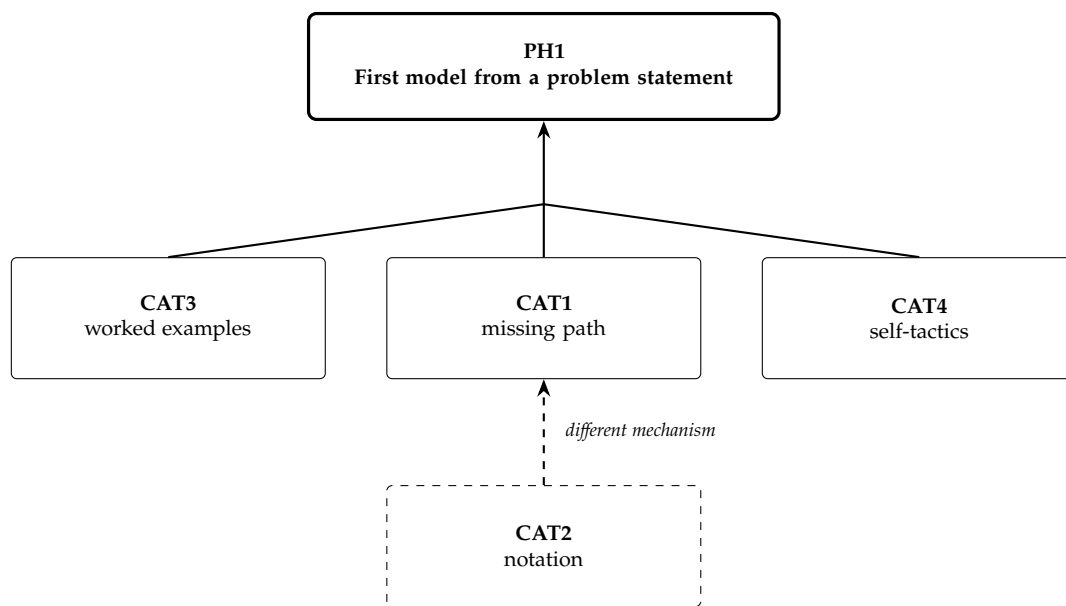


Figure 5.5: PH1 focused view. Feeding categories are CAT1, CAT3, and CAT4. CAT2 is a different mechanism and does not feed PH1.

5.3 RQ2: Strategies and support needs

RQ2 asked what people do about the challenges and what pedagogical or tool support they identify. The primary categories are teacher examples (CAT3), student procedures for getting started (CAT4), when a comment can still be used (CAT5), access to feedback (CAT6), and AI used as an explainer (CAT13).

5.3.1 CAT3 — Teacher examples and walkthroughs unlock work

Worked examples and whiteboard walkthroughs were the main pedagogical unlock. S1 put it directly: “Examples are more useful for me because I learn by seeing things” (S1). L1 described a board ritual: map the statement into a model step by step, then give another exercise to do alone. Wave 2 instantiated the same pattern: S7 recalled a simple board example followed by class debate; S8 said examples taught notation more than the expected answer.

L4 used a different walkthrough. Students solved the exercise with the lecturer, who guided them, rather than the lecturer solving it for them first: “No, I do not solve exercises for them. They solve the exercises with me” (L4). That is another way of guiding the work. The other interviews still describe worked examples and board walkthroughs (S1, L1, S7, S8).

5.3.2 CAT4 — Procedures students used to get started

Students described procedures for getting from the problem to a first diagram: trial-and-error from the text, objects then relations, reading symbols as words, and early drafts. S5’s group scripted the first ten minutes before opening a tool: “Someone reads the text aloud, we each say what jumps out, and we decide who drafts the first diagram” (S5).

Some of these procedures were part of what the class was doing. S7 started quickly “to have something done in case I was the ‘victim’ of the board” (S7). Classmates who had not tried copied from the board. S10 did the exercise in a notebook and then compared it with what appeared on the board. S8 underlined requirements and moved from pencil to pen when describing later class exercises, and said they were already somewhat familiar with making models. That routine is not placed on the bachelor’s project, where S8 did not know how to use the UML the requirements asked for; earlier courses had already used entity-relationship and class diagrams (CAT1). Table 5.3 lists the procedures.

Table 5.3: CAT4 procedures students used to get a first diagram started.

Tactic	Where it showed up
Read the brief aloud, then draft	S5 group script before opening a tool
Objects then relations / symbols as words	Wave-1 process talk (S1–S3)
Start quickly in case of being called to the board	S7
Underline, pencil, then pen	S8, later class exercises; not placed on the bachelor’s project
Notebook, then compare with the board	S10

5.3.3 CAT5 — When a comment can still be used

End-of-course or end-delivery feedback let errors compound. S1 wanted problems caught earlier: “Problems could have been addressed along the way instead of propagating until

the end” (S1). Lecturers often planned several rounds of feedback, but class time and ECTS then cut those rounds short.

S8’s project received walk-around comments that “almost always implied changes, large or small” (S8), with no remembered full redo. Those comments arrived while the work could still be changed. S10 still had time before the next delivery and did “a large local restructuring” (S10). After that, S10 started asking who is external before drawing another BPMN. S8, after a later course, went back over the bachelor’s-project models and found small defects in the syntax and the notations. S7’s grade was given before the comments, and that delivery could not be resubmitted. The comments were for the next phase or the defence. The group then redid the first-phase system from scratch. S7 also described a teammate who rushed and did almost everything. The interviews do not show that the locked grade, rather than that rush, produced the redo. Who could get a comment while the work could still be changed is CAT6.

5.3.4 CAT6 — Feedback access depends on initiative and format

Detailed feedback often arrived only if students asked, and public formats favoured those willing to expose a half-finished diagram. S4 preferred private channels: “Not very comfortable in public. I am shy, so presenting a half-finished diagram to the whole class is stressful” (S4). S5, more outgoing, used a public lab board. Wave 2 widened the conditions beyond a shy/outgoing split: S7 was comfortable showing *models* (more than code) but still treated extra comments as asked-for and in person; S8 shared easily with friends and the teacher, not with course-group strangers; S10 found desk walk-around easy and raising a hand hard: “If the teacher comes by your desk and asks ‘is everything all right?’, it is much easier to show what you are doing than to raise your hand in front of the class” (S10). S9 said that finishing a model before the teacher solved it never happened. Asked what they would do, S9 said they would avoid showing it because the drawing would be poor, not because the model might be wrong. For S8, practice exercises did not count toward the grade. The teacher did not check them, and going to ask was rare. S8 called that a lack of interest, not shyness, though S8 did go a few times. On the project, the teacher checked the work as it went, and those comments arrived while the work could still be changed (CAT5).

Figure 5.6 keeps when a comment arrives (CAT5) and who can get it (CAT6) as two questions. They meet when the comment arrives while the model can still be changed.

5.3.5 CAT13 — AI used to explain or compare

Students who used AI asked it to explain options or compare approaches. S5 also asked a chatbot “to suggest alternative class diagrams from a bullet list, then compared options with the team” and, for graded work, “rewrote everything ourselves and documented changes” rather than handing in the raw output (S5). S10’s incident is the first Portuguese lived case in the corpus: screenshots and small questions after a self-check, never a full

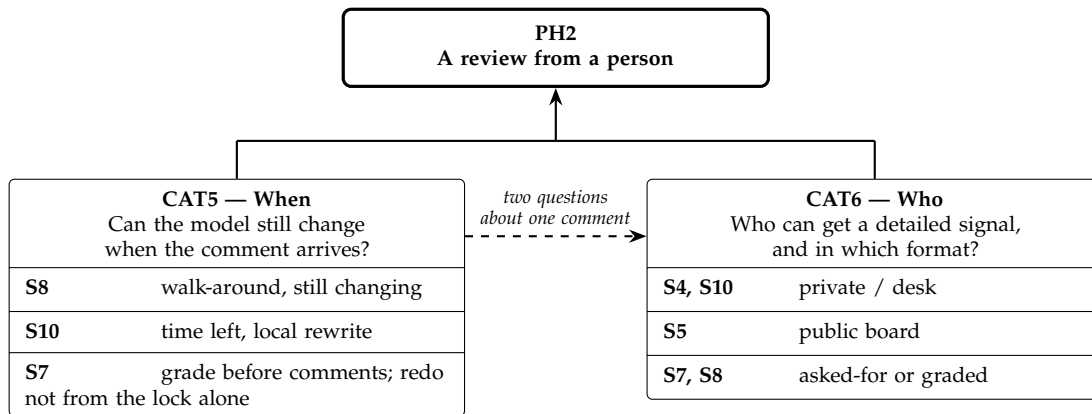


Figure 5.6: CAT5 versus CAT6. When a comment arrives and who can get it are two questions. They meet when the comment arrives while the model can still be changed (S4–S10).

model—“I never asked ‘make me the model of this whole system’.” (S10). That use also substituted for some public questions S10 avoided (CAT6). S4 and S6 reported the same stance, which later feeds PH3 (Section 5.4.3). Figure 5.7 separates what they asked from what they would not hand in. L3 wanted an LLM that would explain a student’s model back in ordinary language, so the student could compare that explanation with what they had meant, rather than a tool that draws the assignment (L3). That is a support need L3 identified. It is not evidence that such a tool works, and it is not one of the student uses in this category. It is not in the figure.

Using the tool is not the same as submitting its diagram

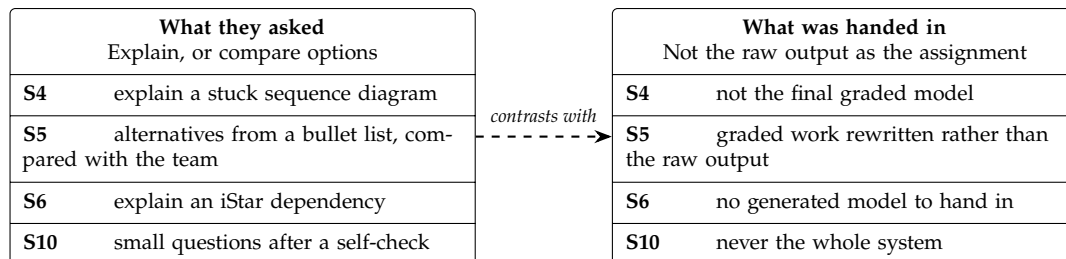


Figure 5.7: CAT13. Students asked a tool to explain a diagram or to compare options. They did not submit a generated model as the graded assignment. S5 asked for alternative class diagrams from a bullet list, compared them with the team, and for graded work rewrote the model rather than handing in the raw output. L3’s wish for an explainer is not shown here.

5.3.6 PH2 — A review often comes from a person

In this study, models and modelling tools often did not compile or simulate execution (CAT12). Students often learned whether a model was good enough from a person: a teacher’s comment, including a late one (CAT5, CAT6). Whether a tool could check the model depended on the tool. Where a checker was available, wanting that check is PH3.

Walk-around, private messages, and public boards change who gets a teacher’s comment in time (Figure 5.8). Students also reviewed models. L4 had groups work the same problem on the board, then swap and validate one another’s work: “It is a kind of peer review” (L4). S4’s own list, and S7’s “going on faith” when an exam answer was not run, are checks students made themselves.

What PH2 is *not*: a claim that automated checkers do not exist, that lecturers refuse to comment, or that students cannot review a model. Checkers appear under PH3 (Section 5.4.3). In these courses, a person was often the review that arrived.

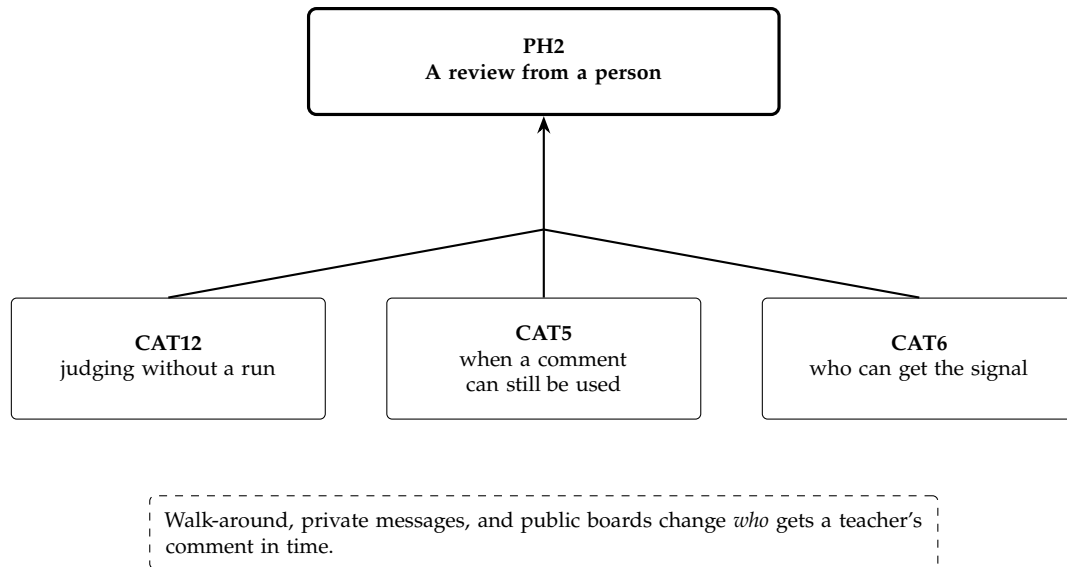


Figure 5.8: PH2 focused view. CAT12 is judging a model without a run. CAT5 and CAT6 name when a comment arrives and who can get it.

5.4 RQ3: Socio-technical conditions

RQ3 asked how tools, course design, domains, and feedback practices shape modelling experience. The categories here are checks and guidance (CAT7), tool friction (CAT8), groups (CAT9), what the model is for (CAT10), and course load (CAT11). PH3 gathers the check and explanation claim from CAT7; CAT8 qualifies that claim and does not feed it. CAT13, reported under RQ2, feeds PH3 as well. Course load leaves less time for demonstrations and feedback. It does not feed PH4.

5.4.1 CAT7 — Checks and guidance are wanted

What was wanted was help with the notation, or a tool that will not accept a model the language does not allow. S3 valued a BPMN interface that “presents a lot of the vocabulary directly in the interface,” explains an object on hover, and can warn when a workflow is connected incorrectly (S3). S3 described that warning as syntax or grammar, not as a check that the diagram fits the situation. L4 valued a modelling tool that provided semantic

checking. In L4's words, "if a student builds a model in piStar, I know the model is semantically correct in terms of the language" and "if the student tries something against the language, the tool does not allow it" (L4). That is L4's claim about the language rules. It is not a finding that the model said what the student meant. L1 preferred a modelling tool over generic drawing when the licence allowed. That is a preference for the kind of tool.

Wave 2 added bounds. S7 saw an open-source compile tool presented and then dropped it: "the interface was a bit antiquated and using it created more problems than it solved" (S7). S8 never used the optional modelling programs in that course, and used a general drawing program. What happened to the tool is reported under CAT8.

5.4.2 CAT8 — Tool friction, switching, and cost

Too many options, licensing, a change of tool during the course, or a large diagram that was slow and came back broken raised the cost of modelling. On a large diagram, S3 saved the file and later found one diagram on top of another, with the page broken. S1 found a modelling tool overwhelming for a simple relation: "The tool had so many options that it became completely overwhelming to know how to make a simple entity-to-box relation" (S1) and preferred a simpler drawing tool. That drawing tool does not check the model. A modelling tool has to present the language, so it cannot be as empty as a blank page. What these accounts add is that several modelling tools were still hard to use. Lecturers reported licensing pushing tool changes (L1). Wave 2 added three different outcomes. S7 tried an outdated compile interface and then dropped it. S8 never used the optional modelling programs in that course, and used a general drawing program. S9 found the UML menus outdated in that company training and kept the tool, because drawing by hand was worse and the tool offered descriptions and templates. Wanting a check (CAT7) did not decide what happened to the tool. Figure 5.9 keeps that wish separate from the cost. It does not include the request for explanations rather than a generated assignment, which is PH3.

5.4.3 PH3 — Want checks and explanations, not auto-drawn homework

PH2 is the everyday situation: a person tells the student whether the model is right or wrong. PH3 is what participants asked for instead. They wanted a check and an explanation, not a tool that draws the assignment (Figure 5.10). CAT7 and CAT13 are part of this phenomenon. CAT8 is not. Wanting a check did not decide what happened to the tool. CAT13 is written up under RQ2. It is named again here because it is one of the accounts that make up PH3.

This is not evidence that a particular checker or chatbot works. L3 asked for a tool that would explain a student's model back. That is a support need L3 stated. It was not tested, and it is not one of the student uses in CAT13. CAT13 rests on four student cases. It is not an evaluation of AI in education.

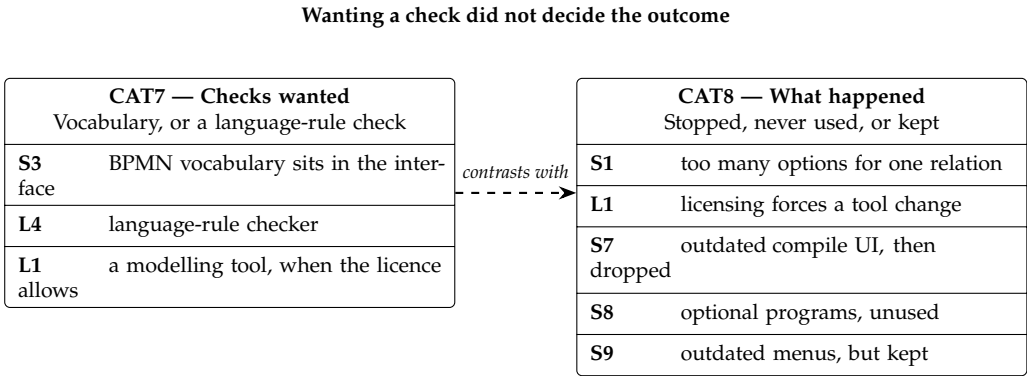


Figure 5.9: CAT7 versus CAT8. Wanting checks and guidance is not the same as keeping the tool that offers them. L1 sits on both sides: a modelling tool is preferred when the licence allows, and licensing can still force a change. S7 dropped a checker after trying the interface. S8 never used the optional suites. S9 kept the outdated tool.

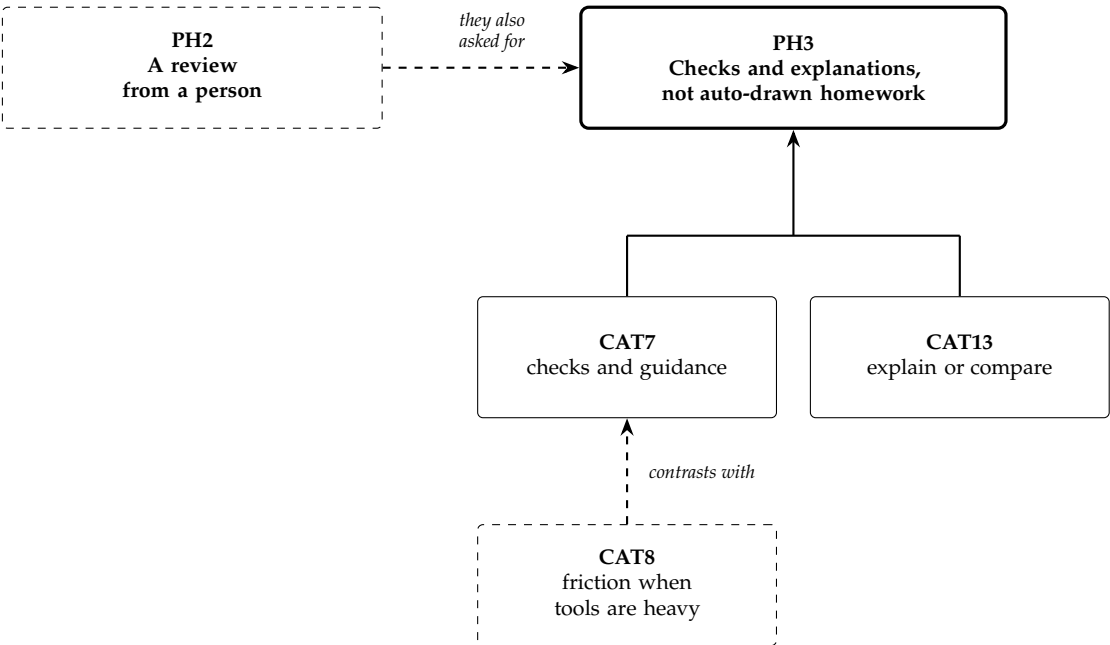


Figure 5.10: PH3 focused view. PH2 is the person-signal they actually get; PH3 is the check and explainer they asked for. Checking is wanted (CAT7); wanting a check did not decide what happened to the tool (CAT8); AI is used to explain or compare; graded work was rewritten rather than handed in as raw output (CAT13).

5.4.4 CAT9 — What the group did with the diagram

S4's group wanted to jump to implementation on a generic shop domain while S4 was still drawing the class diagram. Refinement of that diagram stopped, and it no longer matched the code. A teammate who rushed to finish phase 1 left S7's group to rebuild the system from scratch. The grade had already been given, and that rushed teammate had done most of the first phase. The interviews do not separate those two. S10's group split the work by notation and the diagrams started to disagree: "Then we had two versions of the same system" (S10). They later added a draft-and-review step so everyone could still explain the models at the defence. S10 reported that the diagrams became explainable. The interviews do not include an assessment of whether the models were better. Figure 5.11 shows those three outcomes.

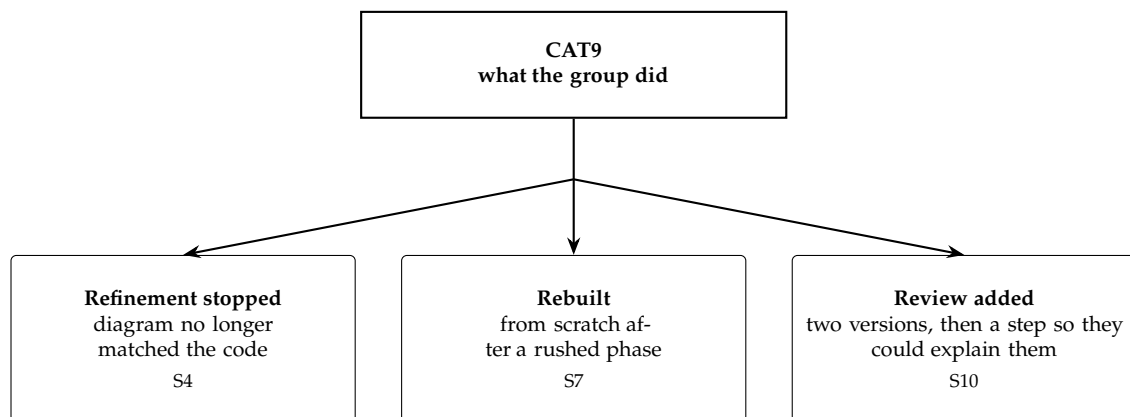


Figure 5.11: CAT9: refinement stopped (S4), the system was rebuilt (S7), or a review was added so the group could explain the diagrams (S10).

5.4.5 CAT10 — What the model is for

S4 said that "Interesting domains make a big difference" (S4). S7 described course models that "would probably only pass through the teacher's hands," and a thesis architecture for a supervisor and a defence that became a priority (S7). The teacher is already a reader. What S7 describes as different is the use: a course submission, or a model that had to explain a system. S10 treated being able to explain the diagram as a defence criterion (CAT12). These accounts do not rank domain interest against having a reader, and they do not show that either one raised the quality of the model. Figure 5.12 shows those reports. It is not the group split in CAT9, and it is not PH4.

5.4.6 CAT11 — Course load and theory–practice gaps

Heavy continuous assessment, missed theory, slide-only catch-up, and late recognition of modelling's value shaped what teaching could actually deliver. Lecturers described workload and continuous evaluation cutting attendance and time for demos and feedback

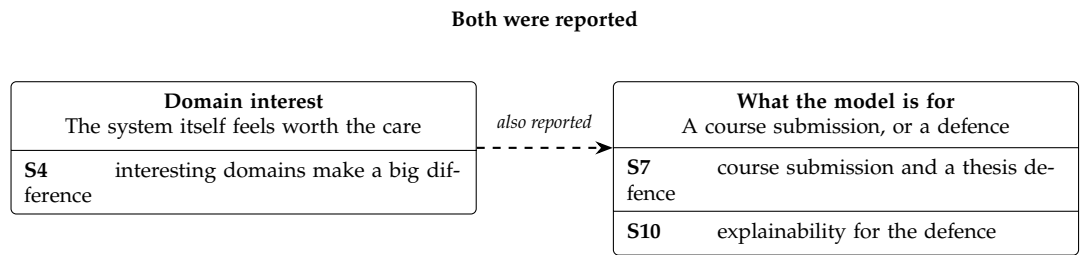


Figure 5.12: CAT10. S4 reported that an interesting domain matters. S7 described a course submission, which a teacher already reads, beside a thesis model for a supervisor and a defence. S10 treated explainability at the defence as a criterion (also CAT12). These influences are not ranked.

(L1, L4). Wave 2 added a student dimension that is not the same as lecturer ECTS-scarcity: S7 tried to finish modelling quickly because a heavy semester left little interest—“because modelling was not something that interested me much, I tried to get those course tasks done as fast as possible so I could focus more on the others” (S7). S8’s paid tutoring job lowered interest more than other courses. When load is this high, there is less time for the demonstrations (CAT3) and the feedback rounds (CAT5) described earlier.

5.4.7 PH4 — Shared work and what the model was for

In the group episodes, what happened to the diagram went together with how the work was shared and what the model was for. S4’s group moved to implementation, and refinement of the class diagram stopped. S7’s group rebuilt the system from scratch after a rushed phase. S10’s group split the diagrams, then added a review so they could explain them at the defence. That is an association in those accounts. It is not a rule that a diagram is kept up only if someone owns it and someone reads it. A course model can still be handed in when the reader is the teacher. A worked example is not, in S4’s account, what kept the group on the diagram after they had moved to code. Course load limits time for demonstrations and comments. It is not a third necessary condition. Figure 5.13 shows how CAT9, CAT10, and CAT11 are drawn on that map.

What PH4 is *not*: a claim that interesting domains do not matter. S4 still held that they do. The interviews do not rank that interest against who will read the model.

5.5 Category summary

Table 5.4 lists each overview category with its research question, a one-line claim, and the memos that most strongly support it. Those identifiers point to the analytical notes reproduced in Appendix B; they are not extra quotations. How categories feed PH1–PH4 is in Figures 5.5–5.13. Four relations are used in this chapter. Examples and a student’s own procedure are ways a first diagram gets started. Difficulty with the notation is a different difficulty from a missing path, and both can occur for the same student. Wanting a check did not decide what happened to the tool. Course load leaves less time for demonstrations

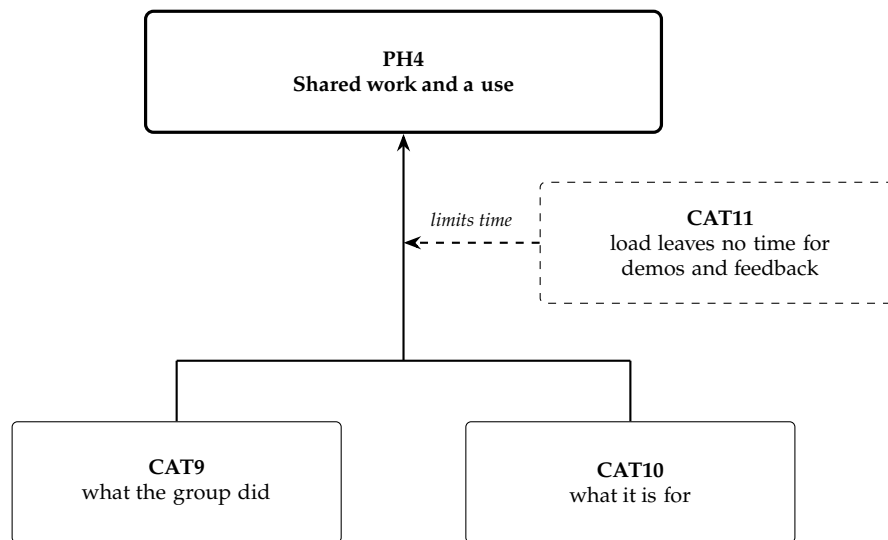


Figure 5.13: PH4 focused view. CAT9 and CAT10 feed the phenomenon. CAT11 limits time for demonstrations and comments. It does not feed PH4.

and comments. It does not block a loop. Other links that were drawn between categories are not claims of this thesis. This is not a finished global theory.

The table is the findings of this chapter. Teaching ideas and open questions that did not earn a category are taken up in Chapter 6 (Table 6.1), not as extra rows here.

Table 5.4: Overview categories, research question, one-line claim, and strongest memos.

ID	RQ	Claim	Memos
CAT1	RQ1	First application is gated on a demo, prior contact, or a project that required notations the student did not know how to use.	S3M1, S8M1, L1M1
CAT2	RQ1	Unfamiliar marks, lookalike diagrams, and a lot to absorb; distinct from a missing path.	S1M1, S3M3, S2M3
CAT12	RQ1	Described tools often did not compile the model; quality is a walkthrough, a justification, and granularity.	S6M2, S4M2, S10M5
CAT14	RQ1	Switching families is a change of question. Leaving a concept out can be right; a missing link or an unclear bridge is a different problem.	S6M1, S10M1, S7M5
CAT3	RQ2	Teacher examples and walkthroughs unlock work (L4 guides the exercise with the students).	S1M2, S7M1, L4M6
CAT4	RQ2	Students described procedures for getting a first diagram started.	S5M3, S7M1, S8M1
CAT5	RQ2	Comments sometimes arrived while the work could still change. A grade before the comments meant that delivery could not be resubmitted. That lock is not shown to have caused S7's redo by itself.	S1M5, S7M3, S10M3
CAT6	RQ2	Access depends on initiative, graded work, and public versus desk format.	S4M3, S8M2, S10M2
CAT13	RQ2	AI is used to explain or compare. Graded work was not handed in as the raw output.	S4M5, S5M4, S10M4
CAT7	RQ3	Checks and guidance are wanted. Wanting a check did not decide what happened to the tool.	S3M4, S7M2, L4M5
CAT8	RQ3	Licensing, option load, switching, and weak UI raise modelling cost.	S1M4, S7M2, L1M5
CAT9	RQ3	A group can stop refining a diagram, rebuild it, or add a review so the members can explain it.	S4M1, S7M3, S10M5
CAT10	RQ3	Domain interest and the use of the model were both reported. They are not ranked, and neither is shown to have raised model quality.	S7M5, S10M5, L4M3
CAT11	RQ3	Load leaves less time for demonstrations and comments.	L1M3, S7M6, S8M3

DISCUSSION

This chapter interprets the 14 categories and four phenomena against the research questions and against related work. What Chapter 5 reports from the interviews stays what the participants said. Where this chapter explains what those reports might mean, it says that the explanation is a reading. An implication is a proposal, and it was not tested. Open questions stay open.

6.1 Answering the research questions

6.1.1 RQ1 — Challenges across modelling activities

The challenges that earned overview categories are not a single “students struggle with UML” story. Starting is gated when a worked path is missing or arrives after a project already required models (CAT1). That gate is distinct from difficulty with the notation itself (CAT2). The UML specification separates concrete syntax, how a concept is drawn, from abstract syntax, the concepts the language allows and the rules for combining them [33]. S1 could name the task and still not know what the marks meant. S8’s requirements diagrams and class diagrams looked alike while standing for different concepts. Moody calls one symbol that stands for several constructs symbol overload [32]. These interviews show unfamiliar marks, similar-looking diagrams, and a lot of information to absorb. They do not show that one symbol was standing for several constructs. In this study, models and tools often did not compile or simulate execution (CAT12). A compile check would show that a model is well-formed, and would leave open whether the diagram says what the person who made it meant. Burgueño, Vallecillo, and Gogolla make that distinction when teaching UML with OCL: a failed check shows that a stated rule was broken, not that the model fits the situation [7]. Switching notation families is a further challenge: locally sensible diagrams lose trace or answer the wrong question (CAT14).

PH1 is the difficulty of making a first model from a written problem statement. Difficulty with the notation is a separate account. S3 said it was the bigger initial problem. The interviews do not rank the two. The interviews do not show that the lecturers in this study failed to teach that step. Four lecturers cannot support that claim. A lecturer may

teach the step, and a student may still not learn it. One reading of the same difficulty is that the student has to understand the problem, decide what this kind of diagram is for, and leave out what that diagram does not need. A class diagram and a goal model do not keep the same details. That reading is an interpretation. It is not what the interviews, by themselves, establish.

This pattern is consistent with published modelling-education work that reports difficulty moving from requirements to models and judging adequacy [26, 31, 10], but it is not a replication of an iStar-only error taxonomy. Li et al. studied novices in one goal-modelling language and derived teaching and tool requirements from that setting [26]. The present corpus spans BPMN, UML, iStar, ER, and courses that teach several of those notations together, and it includes lecturers. The closest empirical overlap is the first-modelling-move problem (PH1) and the demand for support that explains constructs rather than only listing them. Divergence appears where this study splits start-gate from difficulty with the notation, and where CAT14 treats family translation as its own challenge rather than as more notation learning.

CaMeLOT and related Bloom-informed frameworks argue that modelling education under-specifies evaluation and metacognitive outcomes [3, 4, 5]. In this study, students judged a model by walking through it, by justifying each element, or with a list of their own, such as whether every element could be explained, whether the diagram matched the written requirements, and whether it was readable (CAT12, PH2). A teacher's comment was the other way they learned whether it was good enough. A checklist from the instructor might still not be enough. If the list only checks that the model is well-formed, a student can take a correct result as a finished model when the diagram does not yet say what they meant. It can be syntactically in order and still be incomplete. That is a limit of a validity list, not a finding that an instructor's list misled someone in this study. Those frameworks are tools for writing course learning outcomes. This study did not sort interviews into Bloom levels (Chapter 4).

One reading of these checks is that the student took up a way of judging that the class had already shown. S7 said the list used in tests, requirements in ordinary language and then crossed off, might have been what the practicals teacher did on the board. S6 was taught completeness, consistency, and justification, and the personal check was to walk through the model and say why each element is there. S10, after getting a boundary wrong, started asking who is external before drawing the next BPMN, and reviewed the model before asking an AI tool. S3 turned the symbols back into words and checked whether the process in the question came back. Another reading is that the student made a list because nobody had said what counted as good enough, or that the list only ticks off the assignment. S4 had a personal list and said nobody had given it as a rubric. A third reading is that a short or missing list is itself a way of teaching. The student has to make a check that fits how they work, and being able to do that is part of what the course is for. S7's practicals teacher kept saying that more than one model can be right. The interviews do not show that a lecturer left the list out for that reason. S6's personal walkthrough

sat on top of quality ideas that had been taught, so a check of one's own can grow out of teaching as well as out of a gap. The interviews do not show where each list came from, and they do not show that the list made the model better.

6.1.2 RQ2 — Strategies and support needs

Worked examples and board walkthroughs were the main way the work got started (CAT3). L4 used another walkthrough: students solved the exercise with the lecturer, who guided them, rather than the lecturer solving it first. Students also described procedures for getting from the problem to a first diagram (CAT4), including reading the brief aloud, starting quickly in case of being called to the board, and working in a notebook before comparing it with the board. Some of those procedures were part of what the class was doing. The interviews do not show that these courses taught no process. S8 said a course may have explained modelling, and that they either missed it or have forgotten. Feedback is both a timing problem (CAT5) and an access problem (CAT6). Where students used AI, they asked it to explain or compare, and S5 also asked for alternative class diagrams and then rewrote the graded model rather than handing in the generated output (CAT13).

Support needs that participants articulated therefore cluster around (i) seeing a mapping from text to model, (ii) getting a correctness signal while the artefact can still change, and (iii) getting that signal in a format they will actually use (desk walk-around versus raising a hand; private versus public). Tables 6.2 and 6.3 summarise the wave 2 variation behind (ii) and (iii). Research on teaching with modelling tools similarly argues for feedback written for learners, not the cryptic error messages of an industrial compiler [27]. Delayed instructor critique of student diagrams is itself a documented bottleneck, which automated checkers try to shorten [20, 14]. Vemuri et al. report optional BPMN exercises with feedback that still drew few submissions relative to the class [41]. Offering feedback is not the same as students using it. This study adds the access dimension: the same walk-around that helps a reserved student is not the same as a public board that an outgoing student enjoys, and neither is the same as comments that arrive after the grade is locked. Those comments are not only the teacher's. L4 had groups work the same problem and then check one another's models. S10's group later added a review so the members could explain the diagrams. Students also checked their own models. The interviews do not show that every course asked for that review, or that every student did it with care. They do show that a classmate can be the person who looks at the model.

CAT13 should not be over-read as an AI-education finding. Four student cases did not hand in a generated model as the graded work. S5 asked for alternative class diagrams and then rewrote the model. That aligns with PH3 (checks and explanations, not auto-drawn homework) and with assessments of ChatGPT on UML that report limited reliability compared with code generation [8]. It does not evaluate any particular tool. L3's wish for an LLM that explains a student's model back, rather than writing it, is a support need L3 stated, which was not tested, and it is not part of the student uses in CAT13.

One reading of S10's use is a trade-off. S10 made the model, reviewed it, and then sometimes sent a screenshot and asked an AI tool to find inconsistencies or strange notation. S10 checked the suggestions again. If the tool and S10 disagreed, S10 went to the slides, the documentation, or the teacher. S10 also said that small questions to the teacher became less frequent, and that a discussion with the teacher sometimes taught more. The tool kept S10 from staying blocked. It also took the place of some of those questions. Another reading is that checking the suggestion still depends on the student already knowing enough to notice a wrong answer. The interviews do not show that the tool's answer was more reliable than a generated diagram.

6.1.3 RQ3 — Socio-technical conditions

Tools, groups, audiences, and course load co-shape whether modelling happens at all. Checks and guidance are wanted (CAT7). Wanting a check did not decide what happened to the tool (CAT8). A group can stop refining a diagram, rebuild it, or add a review so the members can explain it (CAT9). S4 reported that an interesting domain matters. S7 and S10 described a course submission beside a model they had to explain at a defence. Those reports are not ranked (CAT10). Load and theory–practice gaps leave less time for the demonstrations and the comments that CAT3 and CAT5 describe (CAT11).

Liebel et al. already showed that tool perception depends on usability, the course role of models, and support—not brand alone—and that industrial tools can demotivate when models must generate code precisely and help is unreliable [27]. CAT7 versus CAT8 is the same tension in interview form: participants want checks. S7 tried an outdated compile interface and then dropped it. L1 reported that licensing can still force a tool change. L4 reported that switching tools during the course creates friction. Buchmann et al. argue that models are knowledge structures, not drawings [6]. CAT14's "models answer different questions" is a student-language version of that claim at family boundaries, consistent with teaching materials that treat cross-diagram consistency and UML–BPMN perspectives as learnable content rather than as extra symbols [36, 42]. S6 said that relation had not been explained. S10 was told after the diagram had gone wrong. That is what those two accounts report.

Classroom comparisons of group versus individual UML exercises find that groups are not uniformly better on correctness and completeness [37]. CAT9 does not score correctness. It reports that refinement stopped, a system was rebuilt, or a review was added so the group could explain the diagrams. Those are different outcomes. The group-exercise result does not explain what happened in S4, S7, or S10.

Survey work on domain choice finds that students prefer personally meaningful domains and that educator assumptions about games and technical domains can miss that preference [18]. CAT10 reports both. S4 said an interesting domain makes a difference. S7 and S10 described a course submission, which a teacher already reads, beside a model they had to explain at a defence. The interviews do not separate those influences, and

they do not show that one of them raised model quality.

One reading of these accounts is that the use of the model changed. A teacher already reads a course submission. S7 described course models that would probably only pass through the teacher's hands, and a thesis model that had to explain a system to a supervisor and at a defence. S10 treated difficulty explaining the diagram as a reason to simplify it: if the people who made it could not follow it, it was probably too complicated. S4's check included whether the diagram was readable enough for a supervisor, and S4 also said an interesting domain makes a difference. Another reading is that the extra care came from the grade, from the size of the thesis system, or from the domain. The interviews do not separate those. They do not show that any one of them raised the quality of the model.

Maslov's picture of conceptual modelling education as fragmented across notations describes how that research is published [31]. It does not explain why a student had trouble moving from one diagram to another. CAT14 is that student-level account.

6.2 Implications

The following implications are *not* findings and were not trialled.

Feedback while the grade is still open. A comment can still change the model in front of the student, or it can change how a later model is made. S8's walk-around comments arrived while the project could still be changed. S10 still had time for a large local rewrite, and afterwards started asking who is external before the next BPMN. S10 said that a problem noticed only after delivery can teach the next piece of work, while that piece itself no longer changes. S7's grade came before the comments, so that delivery could not be resubmitted. The comments were for the next phase or the defence. After a later course had taught more notation, S8 went back over the bachelor's-project models and found small defects. The full redo in S7's group is not shown to have been caused by the locked grade alone. If a course has only one modelling submission, a comment during the lab may be the only chance to change that submission. It can still be of use on a later model. That distinction was not tested here.

A question at the desk. A public board helps students who will show an unfinished diagram, and it stops others. A question at the desk can recover work from a student who will not raise a hand and ask. Reserved students ask less often, so they can receive fewer comments than classmates who speak up. S10 found the desk easy and raising a hand hard. Access was also wider than shyness. Some students still treated an extra comment as something they had to ask for, and some shared with friends but not with strangers in the group. This was not tried as a change to a course.

Checks, not generators. PH3 and CAT13 point in the same direction for both modelling tools and AI: validation, explanations, and a tool that checks the syntax or refuses a model

the language does not allow are closer to what participants asked for than a tool that draws the assignment. An experiment with a BPMN drawing tool found mixed results when extra feedback and points were added together [15]. This thesis does not test whether a check would improve the model. L3's proposal belongs here rather than in CAT13: an LLM that explains the student's model back in ordinary language, not one that writes the diagram or the constraints. That reading is consistent with education-tailored feedback arguments in MDSE teaching [27] and should not be confused with a product requirement specification.

Teach the bridge between families. Where several notations are assessed, the difficulty was how the next language relates to the previous one (CAT14). S6 said the notations were known separately, but "the bridge between them was unclear," and that nobody had explained "how much of the iStar model should survive in UML" (S6). S10 had tried to translate BPMN too directly into a class diagram, and was told that the models were answering different questions. S10 said that explanation unblocked the work. S8's requirements diagrams and class diagrams looked similar and still were not the same kind of model (CAT2). An implication is to teach that relation: what question each language answers, and where a direct copy is the wrong move. That teaching was not tested here. Reusing one case study across families is a wish reported in the interviews, not a tested curriculum.

How a new language relates to ones already known. A further implication is how a language just introduced relates to languages the students already know. L4 described models as complementary perspectives: each answers its own question, and together they give a better view of the problem. S6 asked for one continuous example from goals, through quality decisions, to UML, with the trace written down. An implication of those two accounts is to say, for each new language, what it shows on its own and what it shows only together with a language the students already know. That implication was not tested here.

Shared work and what the model was for. In the reported group episodes, what happened to the diagram went together with how the work was shared and what the model was for (PH4). That is not a rule that both are required. Peer review after split-by-notation can surface divergence; it does not by itself create a shared model (S10).

6.3 Limits of the claims

Chapter 4 stated how quality was pursued. This section states what those bounds do to the argument.

The study analyses what participants recounted. It did not watch them model, and it did not assess the diagrams. Recall is uneven. Some episodes were current courses. Others

were years earlier, and S9's degree is recalled weakly. S9's remark about hiding a drawing is what they said they would do, not an event that happened. Some questions offered a word, including a compile button and whether notation was the biggest initial problem. A prompted answer remains the participant's account. Quotations are from cleaned working transcripts. For S7–S10 the English was drafted by an AI tool after identifying details were removed, and it was checked against the Portuguese. The recordings are gone. The thesis therefore does not say what a course taught, does not give one cause for S7's redo, and does not say that a diagram got better. S10 reported that a review made the diagrams explainable. That is not a measured improvement.

The participants were contacts, including course colleagues, and may differ from students who were not asked. The courses, degrees, and languages are not one setting. Claims about teaching time rest on four lecturer interviews from the first round. No lecturer and student from the same course were interviewed, so a lecturer's account of a room that waits is not checked against that room's students. That mismatch is one open question (OQ3). It is not the whole of these limits.

One person analysed the interviews and had studied modelling. That may have introduced biases that were not noticed while the interviews were being conducted. Comparing accounts does not remove that bias. Interviewing stopped because further interviews were hard to arrange and no 15th category appeared. That stop does not settle every relation inside the categories. Chapter 5 states which relations are claimed, and where a claim is thin.

The study did not test a change to a course or a tool. A wanted check, an explanation, or a review step is a reported need or a reported practice. It is not evidence that the change would improve learning. The implications in this chapter are proposals.

Student-side CAT1 and CAT4–CAT6, CAT9, and CAT12–CAT14 are denser after wave 2 than at the wave 1 freeze. No further lecturers were interviewed, so accounts of course load and of planned feedback that could not be kept still come from wave 1. CAT11 therefore remains partly the lecturers' view of institutional constraint, even though S7 and S8 added student load.

Wave 1 left five sampling questions (OQ1–OQ5). They are gaps, not extra categories. No lecturer and student from the same course offering were interviewed, which is why OQ3 is only partly answered: lecturers' "students wait to start" can describe the room without describing every student.

The stop is stated in Chapter 4. After the four further interviews, the 14 categories were judged enough for the research questions of this thesis. Further details could still appear. No further lecturers were interviewed. The study does not claim that the categories were saturated. That stop is not, by itself, evidence that every relation inside the categories is settled.

Smaller leftover questions from individual memos (whether a checker is dropped only when it is optional; whether lab walk-around can replace a second submission; whether exclusive notation ownership was the teacher's design or a group habit) stay as future-work

recipes, not as a recruitment plan for this thesis.

Table 6.1 is the five questions wave 2 was aimed at. They stay as those questions. The table lists what wave 2 did with them, and what remains.

Table 6.1: Open questions that directed wave 2.

ID	Question	Status	What remains
OQ1	Is never reworking a model its own challenge?	Partly answered	Variation sits in CAT5 (when a comment can still be used, and what a closed delivery still leaves open). How common a full redo is still open. S7's redo is not shown to have been caused by the locked grade alone.
OQ2	Do shy and outgoing students need different public-feedback designs?	Answered	CAT6 is wider than shy versus outgoing: access also depends on graded work and on desk versus public format.
OQ3	Do lecturers' "students wait to start" claims match what students say?	Partly answered	Can be true of the room, not of every student. No matched lecturer–student pair in the same course.
OQ4	Is switching between notations more than one RE case?	Partly answered	CAT14 includes S6, S7, and S10. S10 was told afterwards that the models were answering different questions, and that explanation unblocked the work. What remains open is whether these courses had explained that relation before the diagram went wrong. S6 called that relation a bridge and said it had not been explained. The interviews do not leave open whether different languages serve different purposes.
OQ5	Do students describe an abstraction deficit the way lecturers do?	Not answered	Students were not asked whether they lacked abstraction, and none described themselves that way. S6 talked about how much detail is enough, and about how much of an iStar model should survive in UML. S10 was told that a process model and a class diagram answer different questions. Names and requirements carried over. The structure of the process did not. Those are decisions about what a diagram leaves out. They are not a student report of an abstraction deficit. That deficit stays a lecturer interpretation.

The wave 2 densification of CAT5 and CAT6 is easier to scan as two compact tables than as a second reading of Chapter 5. Table 6.2 records three rework modes on the same timing-and-stakes mechanism. Table 6.3 records access conditions that are wider than a shy-versus-outgoing split. Neither table introduces a new category.

6.4 Future work

A later study can take up the remaining questions without pretending that this master's project can still collect that sample. Table 6.1 already labels what wave 2 closed, what it only partly answered, and what it left untouched. The items below are recipes for a different project, not a recruitment plan for this thesis.

Table 6.2: What happened after comments (wave 2). Not every comment was too late to use.

Case	Rework mode	Condition
S8	Local changes (no remembered full redo)	Walk-around comments on a still-open individual project
S10	Large local rewrite	Time remaining before the next staged delivery
S7	Full redo of the first-phase system	Grade before comments, and no resubmit of that delivery; a rushed teammate had done most of the first phase. The redo is not attributed to the lock alone

Table 6.3: CAT6 access conditions. Extra comments still tend to be asked-for; the format of the channel varies.

Case	Access pattern
S4	Shy; prefers a private teacher channel over a public half-finished diagram
S5	Outgoing; uses a public lab / group board
S7	Comfortable showing models (more than code); extra comments still in person and asked-for
S8	Easy with friends and the teacher, not with course-group strangers; ungraded work made checking optional
S9	Hypothetical: would hide a poor drawing rather than a wrong model; S9 said this never happened
S10	Reserved starter; desk walk-around easy, raising a hand is not

- Observe the first minutes of a board laboratory, or interview the lecturer of the same offering as the students, to test wait-to-start as a room-level versus person-level claim (OQ3 remainder).
- Ask, in a course that assesses more than one language, whether students were shown how the next language relates to the previous one before they had to move from one language to the other (OQ4 remainder).
- Collect non-leading incidents about choosing granularity and what to omit, or observe the same task, before treating an “abstraction deficit” as a student finding (OQ5).
- Compare other groups in the same systems-analysis setting on whether a full redo is common or was tied to one rushed teammate (S7), and whether exclusive notation ownership was the teacher’s design or a group habit (S10).

Why interviewing stopped is stated in Chapter 4. That judgment is not a claim that later studies of other countries, notations, or lecturer practice are unnecessary.

CONCLUSION

This thesis asked how software modelling is experienced, taught, and learned in higher education, treating modelling as a socio-technical activity rather than as notation drill alone. Fourteen interviews (four lecturers, ten students) were analysed with a lightweight STGT procedure: open coding, short memos, constant comparison, and a findings map. The result is an empirically grounded account of **14 categories** and **four phenomena**, sufficient for the research questions of this MSc. The four phenomena in Chapter 5 are claims that several categories support together (Figure 5.1). PH1 is the first diagram. PH2, PH3, and PH4 bring together strategies, support needs, and course conditions.

7.1 Answers to the research questions

On RQ1, the main challenges are a gated first modelling move, difficulty with unfamiliar or lookalike notation as a distinct problem, difficulty judging “good enough” when the modelling tools the participants described often did not compile or simulate the model, and breakdowns when students switch notation families. A compile check would show that a model is well-formed, and would leave open whether the diagram says what the person who made it meant.

On RQ2, worked examples and guided walkthroughs remain a main way the work starts. L4 guides students as they solve the exercise, rather than solving it for them first. Students also described their own procedures for getting a first diagram started. The interviews do not show that these courses taught no process. A comment could still change the open work, and who could get it depended on the channel. In one case the grade came before the comments, so that delivery could not be resubmitted. Where students used AI, they asked it to explain or compare, and S5 also asked for alternative class diagrams and then rewrote the graded model rather than handing in the generated output.

On RQ3, checks and guidance are wanted. Wanting a check did not decide what happened to the tool. A group can stop refining a diagram, rebuild it, or add a review so the members can explain it. S4 reported that an interesting domain matters. S7 and S10 described a course submission beside a model they had to explain at a defence. Those

reports are not ranked. Heavy workloads and continuous assessment leave less time for worked examples and for feedback while the model can still change. L1 said classes are not long enough to cover every example, and that students sometimes skip a class to study for a test.

7.2 Contributions

The contribution is a bounded, traceable map of modelling-education challenges and support needs that connects student and lecturer voices to the socio-technical conditions of courses, tools, groups, and feedback. Relative to method-specific error taxonomies and Bloom-based learning-outcome frameworks, the account splits mechanisms that are often collapsed (a missing path versus difficulty with the notation, which are not ranked; when a comment can still be used versus who can get it; checks and guidance, which did not decide what happened to the tool) and treats a change of notation family as its own difficulty, including that leaving a concept out can be the right move. Chapter 6 reads these patterns as possible changes to courses and tools. This study did not test whether those changes would help. The interviews do not show that lecturers fail to teach, comment, or relate one language to another. A comment can still leave the model unchanged. A student can still miss how the next language relates to one they already know.

7.3 Limits

Interviewing stopped after the four further interviews. They did not add a 15th overview category. The study does not claim that the categories were saturated. That stop is not, by itself, evidence that every relation inside the categories is settled. Further details could still appear. The second round was sought because open questions remained. The students who took part had done software modelling and were available. They were not chosen one by one to match each open question. The limit on arranging further interviews is stated in Section 4.6. No further lecturers were interviewed. Several open questions remain labelled for later work (Table 6.1). What transfers is the set of conditions under which a category appeared. What those procedures do to particular claims is stated in Chapter 6.

7.4 Closing

Future work can take up three questions this thesis leaves open. One is whether “students wait to start” describes a whole class or only some students in it. That would need the lecturer and the students from the same course, or a look at the first minutes of a lab. Another is whether, in a course that uses more than one language, students were shown how the next language relates to the previous one before they had to move from one language to the other. A third is whether students, in their own words, describe difficulty with abstraction as a lack they have. Lecturers in this study described that difficulty.

Students did not describe themselves that way. S6 and S10 did talk about what a diagram should leave out, and about how much detail is enough. The detail is in Section [6.4](#). Those questions are a different project from the account this thesis has already produced.

PILOT INTERVIEW GUIDES

This appendix provides the semi-structured interview guides used in the pilot phase and retained, with flexible probing, for the main corpus. The guides are designed to elicit: (i) perceived challenges across modelling activities (RQ1), (ii) strategies for dealing with these challenges and articulated support needs (RQ2), and (iii) socio-technical shaping factors (RQ3). Questions are asked flexibly; probes are used selectively depending on time and participant responses.

Wave 2 student interviews (S7–S10) used the same student guide. Extra probes on start-gate, rework, feedback access, groups, cross-notation switching, and course load were used where they spoke to open questions from wave 1. They are a sampling aid, not a second protocol.

A.1 Instructor pilot interview guide (20–30 min)

0) Opening (30–45s)

Script (say verbatim).

Thanks for meeting with me. I'm studying how modelling is taught and learned in higher education, focusing on challenges, coping strategies/support needs, and how context (tools, course design, domains, feedback) shapes the experience. With your permission, I'd like to record so I don't miss details. The interview should take about 20–30 minutes. You can skip any question.

1) Teaching context (3–4 min)

Q1. Teaching background and where modelling fits.

- *Main prompt:* Could you briefly describe your teaching context and where modelling fits into your courses?
- *Probes (optional):* courses/level; modelling families (UML/BPMN/ER/iStar/MDE); years taught; class size; team vs individual work.

2) Learning goals and course design (6–7 min)

Q2. Intended modelling competence.

- *Main prompt:* When a student finishes your course, what should they be able to do with modelling?
- *Probes:* key activities (interpret/create/refine/evaluate); minimum vs excellent; what “good enough” looks like.

Q3. Scaffolding from problem statement to model (and better model).

- *Main prompt:* How do you scaffold students from the problem statement to a model and a better model?
- *Probes:* explicit step-by-step approach; where students get stuck; quality criteria taught explicitly vs via feedback; reflection/comparison tasks.

Q4. Assessment focus across modelling activities.

- *Main prompt:* How do you assess modelling competence in your course?
- *Probes:* final artefact vs process/reasoning; which activities assessed; quality dimensions graded (completeness/consistency/readability/fit-for-purpose).

3) Observed student challenges (6–8 min)

Q5. Biggest recurring challenges.

- *Main prompt:* What are the most common challenges students face in modelling?
- *Probes:* conceptual (semantics) vs procedural (how to build/refine); which activity is hardest; differences by notation; beginner vs advanced differences.

Q6. Concrete incident walkthrough (high value).

- *Main prompt:* Can you recall a specific moment when students struggled with modelling? What was happening?
- *Probes:* what exactly stuck; how you noticed; conditions (tool, time pressure, group work, unclear requirements, domain unfamiliarity); redesign ideas.

Optional mini-elicitation (2–3 min)

Prompt.

- Could you briefly describe one modelling assignment/exam task you use, and what a strong vs weak submission looks like?
- *Probes:* which activities it tests; diagnostic mistakes; what feedback/support would prevent them.

4) Coping strategies and support needs (6–7 min)

Q7. What you do when students struggle.

- *Main prompt:* When students struggle, what do you typically do to help, and what seems to work best?
- *Probes:* worked examples/templates/checklists/peer review; tailoring by difficulty type; what students do effectively/ineffectively.

Q8. Tool-related supports and constraints.

- *Main prompt:* What role do tools play in these challenges, and what tool support would be most helpful?
- *Probes:* syntax checking, hints, layout/views, versioning/collab; tool friction (setup/usability/cognitive load); tool changes due to student experience.

Q9. Feedback and iteration.

- *Main prompt:* How do you provide feedback on models, and how much opportunity do students have to revise?
- *Probes:* one-shot vs iterative; explicit quality criteria; approaches that improved students' evaluate/refine ability.

5) Socio-technical shaping factors (3–4 min)

Q10. Domains and authenticity.

- *Main prompt:* How do you choose assignment domains/examples, and how do those choices affect engagement or performance?
- *Probes:* familiar vs unfamiliar domains; allowing domain choice; demotivating/excluding domain choices.

6) Closing (2–3 min)

Q11. Ideal supports / recommendations.

- *Main prompt:* If you could change one or two things to improve modelling learning in your context, what would you change?
- *Probes:* course design/scaffolding/assessment; feedback; tool features; “small change, big impact”.

Q12. Anything missing.

- *Main prompt:* Is there anything important I didn't ask but should have?

Close (say verbatim).

Thank you — this was very helpful. If you're interested, I can share a short summary of findings later, and I may follow up with one or two clarifying questions.

A.2 Student pilot interview guide (20–30 min)

0) Opening (1–2 min)

Script (say verbatim).

Hi! Thanks for taking the time. I'm doing a thesis on challenges in learning and doing software/conceptual modelling in higher education. This will take about 20–30 minutes. There are no right or wrong answers—I'm interested in your experience. With your permission, I'd like to audio-record so I don't miss details. The recording will be anonymised and used only for research. You can skip any question, and you can stop the interview at any point. Is it okay if I record?

1) Background and context (3–4 min)

Q1. Studies and modelling exposure.

- *Main prompt:* Can you tell me a bit about your studies and where modelling fits in (programme, year, courses)?
- *Probes:* which courses and purposes; notations used; modelling outside courses (work/internship/personal projects).

2) Experience with modelling (6–7 min)

Q2. Personal meaning of modelling.

- *Main prompt:* When you hear “software modelling” or “conceptual modelling”, what comes to your mind?
- *Probes:* first example/image; useful work vs extra documentation; 1–2 sentence explanation to a peer.

Q3. Useful moment story.

- *Main prompt:* Can you remember a moment when modelling felt particularly useful?
- *Probes:* context/task; what it helped (understand/decide/communicate); effect on confidence/motivation.

Q4. Frustrating moment story.

- *Main prompt:* Can you remember a moment when modelling felt confusing, pointless, or frustrating?
- *Probes:* cause (notation/tool/task/feedback/group/time); coping response; what would have helped.

3) Learning and challenges across modelling activities (7–8 min)

Q5. Learning a new notation/framework.

- *Main prompt:* When you learn a new notation/framework, what's usually the hardest part at the beginning?
- *Probes:* symbols vs semantics vs how to start; specific notation; strategies used to learn.

Q6. Creating a model from a text description.

- *Main prompt:* Imagine you get a text description and have to build a model. What steps do you take?
- *Probes:* starting point; structured strategy vs trial-and-error; what to include/omit; whether taught a step-by-step approach.

Q7. Deciding whether a model is “good enough”.

- *Main prompt:* Once you have a model, how do you decide whether it's good enough or needs changes?
- *Probes:* criteria (completeness/consistency/readability/fit); quality rules/checklists; comparing with peers.

4) Socio-technical factors (6–8 min)

Q8. Tools.

- *Main prompt:* What tools have you used for modelling, and how has that been?
- *Probes:* liked/disliked tools and why; tool makes modelling harder/easier; whether tools helped catch mistakes.

Q9. Assignments, domains, authenticity, and inclusion.

- *Main prompt:* What kinds of domains/examples did you work with in modelling assignments?

- *Probes*: motivating vs alien domains; domain choice; gentle inclusion probe (if appropriate).

Q10. Feedback and iteration.

- *Main prompt*: How did feedback on modelling work in your courses?
- *Probes*: type/timing of feedback; what would have helped most; effect of group work on modelling.

5) Closing (3–4 min)

Q11. “Good modeller” competence.

- *Main prompt*: What should a good modeller be able to do?
- *Probes*: habits/abilities; soft skills vs technical; whether courses helped develop them.

Q12. Improvements and support needs.

- *Main prompt*: If you could change one or two things in how modelling is taught, what would you change?

Final open question.

- Is there anything important about learning or using modelling that we didn’t cover?

A.3 Mapping of interview questions to research questions

Table A.1 summarises how the main interview prompts operationalise the analytic focus of each research question. Probes are used flexibly to adapt to participant experience and time constraints.

Table A.1: Mapping of pilot interview guide questions to research questions.

Guide	Question (main prompt)	RQ1	RQ2	RQ3
Student	Q1 Background and modelling exposure			✓
Student	Q2 Meaning/perception of modelling	✓		✓
Student	Q3 Useful moment (incident)	✓	✓	✓
Student	Q4 Frustrating moment (incident)	✓	✓	✓
Student	Q5 Learning a new notation/framework	✓	✓	✓
Student	Q6 Creating a model from text (procedure)	✓	✓	
Student	Q7 Judging quality / “good enough”	✓	✓	
Student	Q8 Tools	✓	✓	✓
Student	Q9 Domains/authenticity/inclusion		✓	✓
Student	Q10 Feedback and iteration	✓	✓	✓
Student	Q11 “Good modeller” competence	✓		✓
Student	Q12 Improvements / supports		✓	✓
Instructor	Q1 Teaching context and where modelling fits			✓
Instructor	Q2 Intended modelling competence	✓		✓
Instructor	Q3 Scaffolding from problem to model	✓	✓	✓
Instructor	Q4 Assessment focus and criteria	✓	✓	✓
Instructor	Q5 Recurring student challenges	✓		✓
Instructor	Q6 Concrete incident walkthrough	✓	✓	✓
Instructor	Q7 Instructor interventions and what works		✓	✓
Instructor	Q8 Tool supports and constraints		✓	✓
Instructor	Q9 Feedback and revision opportunities	✓	✓	✓
Instructor	Q10 Domains/authenticity effects		✓	✓
Instructor	Q11 Ideal supports / recommendations		✓	✓
Instructor	Q12 Anything missing (catch-all)	✓	✓	✓

ANALYTIC AUDIT TRAIL

This appendix reproduces a *selection* of analytical memos and open-code notes, not the full first-cycle corpus (277 codes, 78 memos). The selection is the memos named in Table 5.4 plus one code note per major cluster and the excerpts used in Table 4.6. Portuguese interviews are quoted from the English translation used in analysis. Those working transcripts were cleaned by the author. The English text of S7–S10 was drafted by an AI tool after identifying details had been removed from the Portuguese, and the author compared the two side by side. The recordings have been deleted, so the Portuguese original is the remaining check for S7–S10. An English-language interview can be checked only against its cleaned transcript. Identifying course titles were already generalised in the vault notes.

B.1 How to read the cards

Each **memo** card stores one analytic idea. Fields: identifier (CaseMn), case, linked open-code IDs, transcript turn IDs (T00n), the idea, and why it mattered for category development. Memos are not findings by themselves; categories in Chapter 5 rest on comparison across memos.

Each **code** card stores one open code: a short title, case, turn IDs, and an excerpt from the cleaned working transcript. Codes stay close to the participant account. Student evidence, lecturer interpretation of student behaviour, and stated support needs were kept distinct in later category work. A need a participant stated is a finding; whether it would work was not tested (Chapter 4).

Turn identifiers mark substantive transcript turns. They are not timestamps and were not invented after the fact.

B.2 Index of reproduced memos

Negative or boundary memos that also appear above include S5M3 (can start without waiting) and L4M6 (guides the exercise with the students).

Table B.1: Memos reproduced below, by overview category (same identifiers as Table 5.4).

CAT	RQ	Memos in this appendix
CAT1	RQ1	S3M1, S8M1, L1M1
CAT2	RQ1	S1M1, S3M3, S2M3
CAT12	RQ1	S6M2, S4M2, S10M5
CAT14	RQ1	S6M1, S10M1, S7M5
CAT3	RQ2	S1M2, S7M1, L4M6
CAT4	RQ2	S5M3, S7M1, S8M1
CAT5	RQ2	S1M5, S7M3, S10M3
CAT6	RQ2	S4M3, S8M2, S10M2
CAT13	RQ2	S4M5, S5M4, S10M4
CAT7	RQ3	S3M4, S7M2, L4M5
CAT8	RQ3	S1M4, S7M2, L1M5
CAT9	RQ3	S4M1, S7M3, S10M5
CAT10	RQ3	S7M5, S10M5, L4M3
CAT11	RQ3	L1M3, S7M6, S8M3

B.3 Memo cards

S3M1 — earlier BPMN contact unlocks MSP start

Case: S3. **Codes:** S3-C04, S3-C05, S3-C06, S3-C17. **Turns:** T013, T017, T019, T043.

Idea. S3 encountered BPMN in two brief earlier courses before MSP. That prior contact let him skip basic assembly and start exercises as soon as they appeared, while a peer without that history needed more kick-start help. Crash-course conditions (about one week of BPMN, then the course moved on) built familiarity but not depth; MSP then added specific symbols and terminology on top.

Why it matters. For STGT, repeated socio-technical exposure—notation plus tool plus short practical windows—changed what “starting to model” felt like in the main modelling course. Prior contact with the same tool/notation pair (Bizagi + BPMN) reduced dependence on waiting for a board example.

S8M1 — modelling arrived as a project requirement first

Case: S8. **Codes:** S8-C01, S8-C03, S8-C04. **Turns:** T003, T011, T013.

Idea. S8 first had to produce UML diagrams in the bachelor’s project without knowing the notations. The underline, pencil, and pen routine belongs to later class exercises, not to that project. Teacher examples in those later exercises mainly taught notation, not the expected answer.

Why it matters. Starting without knowing the notation can happen as a project requirement. The later sketching routine is a separate account.

L1M1 — breaking the ice is the main teaching hurdle

Case: L1. **Codes:** L1-C03, L1-C04, L1-C07, L1-C11. **Turns:** T012, T014, T022, T024, T037.

Idea. L1 ranks “breaking the ice”—actually starting to apply modelling to solve a problem—as harder than absorbing semantics alone. He sees most students wait for the teacher to start the solution; only a minority begin independently. Early work involves many typical notation errors until students practise mapping requirements into a model.

Why it matters. Lecturer interpretation centres procedural start-up, not just concept difficulty. This is a pedagogy–culture–confidence bundle: students, teaching moves, and expected teacher role interlock before tool or assessment details dominate.

S1M1 — notation and tool overload block starting

Case: S1. **Codes:** S1-C07, S1-C08, S1-C12. **Turns:** T018, T020, T029, T051.

Idea. On the FreeCol project, dense notation and a feature-rich tool stacked together. Many symbol choices created doubt about the "proper" arrow or relation before any modelling could begin. StarUML's interface felt like too much at once for simple entity-to-box work.

Why it matters. STGT: the difficulty is not cognition alone but a socio-technical bundle—course task, UML notation, and StarUML options jointly raised the entry cost. Tool design and notation load shape whether students can start at all.

S3M3 — notation detail stalls otherwise smooth progress

Case: S3. **Codes:** S3-C10, S3-C11, S3-C12. **Turns:** T025, T027.

Idea. S3 describes BPMN as broadly intuitive until notation details appear: duplicate gateway symbols, expanded subprocesses, unfamiliar constructs, or visually dense diagrams where sequence flow is hard to track. Progress can feel fine until a new symbol or layout problem stops work and triggers re-questioning.

Why it matters. Difficulty is not only "learning BPMN" once—it recurs at notation boundaries and at the visual/tool layer when models grow. This aligns with a socio-technical view where symbol set, diagram layout, and reading complexity interact.

S2M3 — framework burden in course not elsewhere

Case: S2. **Codes:** S2-C08, S2-C09, S2-C30, S2-C07. **Turns:** T022, T024, T026, T103, T106.

Idea. Course modelling felt frustrating because unfamiliar frameworks had to be learned before drawing, and abstractions had to fit FreeCol's existing rules. At work and in the dissertation, informal intuitive diagrams sufficed; prescribed frameworks and their multiplicity are off-putting.

Why it matters. STGT: the same person experiences modelling differently across contexts—course (framework + legacy codebase) versus self-directed work (purpose-built sketches). Institutional notation requirements may deter practice even when the student values abstraction.

S6M2 — a walkthrough and a justification, not a compile check

Case: S6. **Codes:** S6-C05, S6-C09, S6-C15. **Turns:** T013, T021, T037.

Idea. S6 experiences requirements modelling as prolonged ambiguity: there is no compile button, so correctness comes from walking through the model and explaining why each element exists. The course required justification for actors, dependencies, and goal-to-class mappings; vague briefs and role-play interviews showed requirements shift with who you ask. Initial stress from indefiniteness later read as intentional pedagogy.

Why it matters. STGT highlights how evaluation norms (justification, traceability, stakeholder interviews) constitute the "technical" environment alongside tools. Modelling discomfort is socio-epistemic—living without executable feedback—not merely unfamiliar syntax.

S4M2 — worked examples unlock reasoning and granularity

Case: S4. **Codes:** S4-C03, S4-C04, S4-C05, S4-C06. **Turns:** T009, T011, T013.

Idea. The student can start from text alone but feels less confident without a teacher example, especially when task style is ambiguous. Examples that explain relation choices and contrast too-detailed versus too-abstract diagrams teach process, not just final shapes. A personal quality checklist grew from draft feedback rather than an official rubric.

Why it matters. Pedagogical artifacts (explained examples, feedback loops) scaffold judgment about "good enough"—a teaching-tool-learner interplay central to STGT.

S10M5 — exclusive notation ownership split the system

Case: S10. **Codes:** S10-C01, S10-C12, S10-C13. **Turns:** T005, T047, T049, T051, T057.

Idea. Splitting BPMN / requirements / UML by person produced two versions of the same system. The group invented draft-plus-review and a short carry-over talk before the next model, because the defence asked people individually. Explainability became a practical quality test: if the three authors could not walk it, it was too loaded. Thesis work later dropped strict UML in favour of communicating the architecture.

Why it matters. Same split-by-notation risk S7 feared; S10 actually saw divergence, then built the peer-review fix S7 described. The teacher is already a reader. What S10 describes is having to explain the diagram: if the people who made it could not follow it, it was too complicated.

S6M1 — multi-notation RE breaks at translation gaps

Case: S6. **Codes:** S6-C01, S6-C02, S6-C08, S6-C14. **Turns:** T007, T019, T035.

Idea. S6's hardest moment was translating an iStar rationale model into UML: the class diagram looked syntactically fine but lost goal traceability because nobody explained how much strategic content should survive. Each notation family (iStar, NFR, UML) demands a different mindset—organisational/political versus structural—and restarting is expected in a multi-family course, but the missing translation methodology is the real pain. S6 wants one continuous worked example from goals through NFR decisions to UML with documented trace links.

Why it matters. Socio-technical RE pedagogy is not only teaching separate notations but orchestrating transitions between them. Without explicit bridge procedures and integrated artefacts, students produce locally correct diagrams that fail globally as requirements models.

S10M1 — BPMN-to-UML breaks when treated as the same model

Case: S10. **Codes:** S10-C02, S10-C03, S10-C04, S10-C05, S10-C06. **Turns:** T007, T009, T013, T015, T017, T019, T021.

Idea. S10's stuck moment was not a missing symbol. A payments service was clear as an external BPMN pool; a too-direct translation into UML left no place for it (actor, class, component, or omit). The teacher's after-class point was that models answer different questions. Names and requirements carried; gateways, lanes, and tasks did not become classes. One-to-one mapping produced action-classes; reasoning had to restart even though domain knowledge stayed. Traceability was mentioned, not practised across models.

Why it matters. This densifies CAT14 at IST AMS beyond S7's syntax-friction account, in the same family as S6's lost iStar-to-UML trace: locally sensible diagrams fail at the bridge. S10 names the missing method in student language ("different questions"), not only as a later teaching wish.

S7M5 — new syntax at the switch; a thesis model to explain

Case: S7. **Codes:** S7-C01, S7-C14, S7-C15. **Turns:** T005, T033, T039, T041.

Idea. S7 did not call modelling difficult. Confusion was duplicated teaching of the same idea (Databases vs class diagrams) and syntax when moving families (relation types). Once already modelling, the next family was easier to start. Coursework models felt like they only passed through a teacher who had seen millions; the thesis architecture for supervisor and defence became a priority. After graduation S7 modelled to explain ideas without worrying about notation rules, and would rather teaching stressed usefulness and fewer families.

Why it matters. CAT14 in this case is light syntax friction plus duplicated course notations, not S6's lost goal-to-UML trace. The teacher is already a reader. What S7 describes as different is the use: a course submission, or a model that had to explain a system. Informal workplace modelling is S7's own report, not a lecturer claim.

S1M2 — worked examples bridge text and diagram

Case: S1. **Codes:** S1-C13, S1-C23, S1-C15. **Turns:** T031, T033, T071, T073.

Idea. S1 learns text-to-diagram mapping by trial and error, starting from components then placing actors. Worked examples—"this description translates to this model"—are named as the most useful feedback form. Templates for common steps would also help.

Why it matters. Without explicit process teaching (see S1-C15), example-driven learning becomes the main scaffold. STGT: pedagogy (examples, templates) substitutes for missing procedural instruction and couples learner strategy to course feedback design.

S7M1 — board labs split starters from copiers

Case: S7. **Codes:** S7-C02, S7-C03, S7-C04, S7-C05. **Turns:** T009, T011, T013, T017.

Idea. IST practicals unlocked work with a simple board example, a class-built requirements list, then a student at the board while the teacher stayed quiet until a student debate finished. S7 started immediately to have something in case of being picked at random, and because not trying drew more teacher displeasure than a wrong attempt. Classmates often waited and copied the board.

Why it matters. The same lab ritual produces two student strategies: attack-to-survive versus wait-to-copy. Lecturer "students wait to start" claims can be true of the room without being true of every student.

L4M6 — active discovery pedagogy resists passivity

Case: L4. **Codes:** L4-C21, L4-C22, L4-C25, L4-C32. **Turns:** T046, T048, T055, T067.

Idea. The lecturer refuses to front-load solved examples, uses icebreakers so shy students cannot wait all semester, and builds solutions gradually from photographed board work while admitting not knowing everything. That contrasts with students overwhelmed by continuous evaluation who rush task-to-task without assimilating.

Why it matters. Teaching stance (discovery, openness, pacing) interacts with course load and classroom social format—relevant when shy students prefer private feedback elsewhere.

S5M3 — early sharing and self-initiated drafts replace waiting

Case: S5. **Codes:** S5-C04, S5-C07, S5-C14. **Turns:** T009, T017, T033.

Idea. S5 rarely waits for a teacher demo: they draft version one, share work-in-progress in lab and group chat, and treat peer feedback as the safety net. Early public sharing reduces anxiety because errors surface early—though S5 acknowledges this pressure could harm shy peers. They want more assessed weekly diagram milestones with peer comments to institutionalise what their team already does informally.

Why it matters. Socio-technical conditions (assessment timing, lab culture, chat channels) shape modelling pedagogy as much as notation teaching. A student who initiates drafts and shares early experiences a different course than one who waits for authoritative solutions.

S1M5 — end-only feedback lets errors accumulate

Case: S1. **Codes:** S1-C22, S1-C18. **Turns:** T065, T067, T069, T043.

Idea. Modelling feedback came mainly at the final project discussion, not iteratively. Problems piled up until they surfaced as "completely wrong" at the end. S1 cannot recall formal validation rules from the course.

Why it matters. STGT: feedback timing and assessment design shape learning loops. Without mid-course modelling checks or memorable quality criteria, students rely on self-judgment and peer/dissertation comparisons instead of course scaffolding.

S7M3 — grade before the comments; a rushed teammate had done most of the first phase

Case: S7. **Codes:** S7-C09, S7-C10. **Turns:** T023, T025.

Idea. The two-phase AMS project gave feedback after a grade, with no resubmit of that delivery—only a chance to fix the next phase or the defence. S7's group then had to rebuild the first-phase system from scratch. A "get-it-done-fast" teammate did almost everything immediately; the two who wanted to plan inherited a rushed model and lost the usual benefit of splitting load.

Why it matters. The grade came before the comments, so that delivery could not be resubmitted. A rushed teammate had done most of the first phase. The interviews do not show that either one, alone, produced the redo.

S10M3 — timed local rewrite taught a boundary self-check

Case: S10. **Codes:** S10-C09, S10-C10. **Turns:** T033, T035, T037, T039.

Idea. Phase feedback caught a mixed BPMN perspective: an extra pool for something inside the organisation, and message flows that should have been sequence flows. With time left before the next delivery, the group did a large local rewrite, not a full abandon. After that, S10 started asking who is external before drawing. Notation nags were memorised; conceptual comments changed later work.

Why it matters. Unlike S7's grade-then-feedback full redo, S10 had time to act and the error was structural, not a rushed teammate. CAT5 is not only "too late": when a loop exists, conceptual feedback can become a new start habit.

S4M3 — shy students need private feedback channels

Case: S4. **Codes:** S4-C07, S4-C08. **Turns:** T015, T017, T033.

Idea. Design ambiguity (multiplicity, class vs attribute) is resolved in office hours or email, rarely in front of the class. Public presentation of half-finished diagrams feels stressful; quick private feedback would enable earlier sharing. The student explicitly asks for complete worked examples in materials so preparation can happen privately before targeted questions.

Why it matters. Social format of courses—public vs private validation—shapes modelling participation as much as notation instruction.

S8M2 — ungraded practice makes checking optional

Case: S8. **Codes:** S8-C02, S8-C05, S8-C06, S8-C07. **Turns:** T007, T009, T015, T019, T021.

Idea. Board examples plus a practice list were not submitted or checked; leaving was allowed. S8 says classmates skipped teacher checks from lack of interest because the exercises did not count, not from shyness. Camunda and Archi were available and unused; draw.io was enough. In the individual project the teacher walked around and almost always asked for changes, but S8 remembers no full redo. Sharing was easy with close friends and with the teacher, not with course-group strangers.

Why it matters. Feedback access (CAT6) here is graded-work gated: initiative plus whether the task counts. Optional semantic tools do not get used when a drawing tool already satisfies “requirements covered.”

S10M2 — reserved starter: notebook yes, board no

Case: S10. **Codes:** S10-C07, S10-C08, S10-C14. **Turns:** T025, T027, T029, T031, T055.

Idea. S10 solved the exercise in the notebook, then compared with the board. Silence was not “I cannot start” and not mainly fear of being wrong; it was dislike of twenty people watching while still thinking. Small arrow-type doubts often stayed unresolved until a classmate, a later search, or the correction. Desk walk-arounds made showing work easy; raising a hand in front of the class did not.

Why it matters. Answers S7M1: this reserved IST student is a starter, not a copier. CAT6 here is public-format gating. The same lab that S7 used for teacher goodwill is, for S10, a place to stay quiet unless the teacher comes to the desk.

S4M5 — tool semantics and workplace informality diverge from class

Case: S4. **Codes:** S4-C01, S4-C12, S4-C13. **Turns:** T005, T023, T025, T031.

Idea. ChatGPT helps explain diagrams in plain language but is rejected for generating graded models. Semantic lab tools catch invalid UML; fast sketch tools allow mistakes at home. A short internship used whiteboard sketches and prototypes—not formal UML—surprising expectations built from coursework.

Why it matters. Tool affordances and workplace norms sit alongside classroom notation teaching; misalignment may reinforce “modelling feels pointless” when industry practice looks informal.

S5M4 — AI and live tools in the group modelling loop

Case: S5. **Codes:** S5-C10, S5-C11, S5-C12. **Turns:** T023, T025, T027.

Idea. S5 uses ChatGPT to brainstorm alternative diagrams and simplify overcomplicated sequence descriptions, then rewrites everything manually before submission. Miro’s live co-editing lowered the barrier for group modelling; Visual Paradigm added stricter semantics before coding. They want semantic consistency checking (“this message does not match any operation”) more than generative box-drawing. Team passivity, not domain excitement, determines whether modelling effort feels worthwhile.

Why it matters. Tools, AI assistants, and team social dynamics form a coupled loop: the same Miro board that enables coordination also needs manual sync when code outpaces diagrams. AI enters as a private brainstorming layer, not a submission shortcut—pointing toward checker-style support rather than replacement modelling.

S10M4 — AI as a distrusted classmate, not a homework generator

Case: S10. **Codes:** S10-C11, S10-C18. **Turns:** T041, T043, T045, T057, T059.

Idea. S10 used AI after a self-check: screenshot plus system context, small questions (external vs actor), what should carry from BPMN, which model next. Never “make the whole model.” It unblocked small doubts and cut some teacher questions, at the cost of missing discussions that taught more.

Why it matters. S10 asked the tool to explain or compare after a self-check, and did not ask it to make the whole model. The interview was in Portuguese. The tool also stands in for the public question S10 avoids.

S3M4 — Bizagi scaffolds vocabulary and syntax

Case: S3. **Codes:** S3-C14, S3-C15, S3-C16. **Turns:** T034, T036, T038, T040.

Idea. Bizagi Modeler supported learning through in-interface vocabulary, hover explanations, and workflow validity checks. S3 found it intuitive from first open and says it clarified flow distinctions. At scale, one-by-one construction and occasional save corruption made the same tool frustrating for thesis-sized diagrams.

Why it matters. Tool affordances are part of the teaching-learning system: they lower vocabulary and syntax entry costs for beginners but do not equally support productivity or reliability at larger model sizes. Tool choice is tied to course reuse across three BPMN contexts and thesis work.

S7M2 — labs are gut feeling; exams are going on faith

Case: S7. **Codes:** S7-C06, S7-C07, S7-C08. **Turns:** T019, T021, T035.

Idea. Students treated the teacher’s pre-drawn board solution as the most correct answer even while the teacher called alternatives a fun fact. In practicals, “good enough” was gut feeling and a tick-off of a homemade requirements list. In tests there was no compile signal, so completeness was fear plus faith. An open-source checker was shown once and dropped because the interface created more problems than it solved; remembered tools were family-specific, weak UI, and mainly for pretty PDF export.

Why it matters. Assessment risk, not notation knowledge, changes how students judge models. Wanting a compile check (CAT7) can coexist with rejecting the actual checker when tool friction (CAT8) outweighs the check.

L4M5 — whiteboard compensates for missing integrated tools

Case: L4. **Codes:** L4-C16, L4-C26, L4-C27, L4-C28, L4-C29. **Turns:** T039, T057, T061, T062, T064.

Idea. Without affordable, integrated, semantics-aware tooling across goal-oriented and object-oriented families, the lecturer relies heavily on whiteboard discussion, manual consistency checks, and family-specific tools like piStar, an iStar tool that does not allow a model the language forbids. Switching tools mid-course creates friction; the top realistic improvement would be a lightweight validator that bridges paradigms.

Why it matters. Tool ecology (licenses, semantics, integration) directly constrains what can be taught, checked, and iterated in class—a core STGT pairing of technique and infrastructure.

S1M4 — simpler tools reduce option overwhelm

Case: S1. **Codes:** S1-C20, S1-C08. **Turns:** T049, T051, T053, T055.

Idea. StarUML made modelling harder through option overload; draw.io felt easier because fewer choices were available. Tool design is seen as capable of improving the modelling experience, separate from notation learning.

Why it matters. STGT links tool affordances to learner engagement: a professional-feeling tool can increase cognitive load for students who are not deeply invested in software engineering. Tool simplification is a socio-technical lever alongside teaching.

L1M5 — proper tools versus draw.io in teaching stance

Case: L1. **Codes:** L1-C17, L1-C18, L1-C20, L1-C21. **Turns:** T057, T059, T061, T063, T071, T073.

Idea. L1 steers students away from draw.io toward metamodel-backed tools (e.g. Visual Paradigm) but struggles with licensing, imperfect checking, and past bad tools he abandoned. Class time is too short for tool onboarding and troubleshooting. He enthusiastically endorses a hypothetical faculty-provided free tool with feedback, checklists, and next-step guidance—explicitly a design preference, not an implemented system.

Why it matters. Tool policy is part of teaching: what counts as "modelling" vs drawing, what errors are caught, and who pays (institution vs student). Lecturer ideal tool features partially overlap student reports (vocabulary help, syntax warnings) but add institutional provisioning.

S4M1 — refinement stopped when the group moved to code

Case: S4. **Codes:** S4-C02, S4-C09, S4-C11. **Turns:** T007, T017, T021.

Idea. In a generic shop group project, teammates pushed Spring implementation while the student maintained the class diagram alone; dismissal as "paperwork" stopped refinement until the model no longer matched code. Modelling worked later when one person owned an interesting library design; early sharing in groups turned diagrams into arguments rather than improvements.

Why it matters. Socio-technical conditions—ownership split, domain interest, group dynamics—can matter more than UML notation knowledge for whether modelling persists.

L4M3 — domain freedom trades lecturer workload for fairness

Case: L4. **Codes:** L4-C06, L4-C07, L4-C08, L4-C11. **Turns:** T026, T027, T028, T033.

Idea. Letting groups choose or refine domains—scoped when too simple or complex—increases marking and feedback work but avoids imposing uninterested domains the lecturer remembers as unfair. Modelling is reserved for problems complex enough to need communication beyond a weekend build. Under semester overload, some students still ask for tighter "just tell me what to do" tasks.

Why it matters. Course conditions (freedom, fairness, workload, concurrent evaluations) co-shape whether modelling feels worth the effort for both teacher and learners.

L1M3 — missing theory class breaks slide-only catch-up

Case: L1. **Codes:** L1-C14, L1-C15, L1-C22. **Turns:** T047, T051, T077.

Idea. L1 reports students skipping theoretical sessions, then learning from slides alone, misinterpreting concepts, and arriving at practicals unprepared. He does not repeat theory in labs and resists making slides a full tutorial. He attributes absences partly to Portuguese student culture, heavy multi-course workload, and continuous evaluation pressure—not lack of interest in modelling alone.

Why it matters. Course structure assumes theory attendance; practicals "just use" concepts. Socio-organisational conditions (semester load, evaluation design) undermine that assumption and create a theory–practice gap independent of notation choice.

S7M6 — competing courses made modelling a task to finish quickly

Case: S7. **Codes:** S7-C16. **Turns:** T013, T035.

Idea. S7 called the modelling workload adequate, but a packed second-year semester plus low interest meant finishing those tasks fast to protect other courses. Drawing and modelling were not a strength or a focus. That condition sits beside the board-lab split: some students only copied, and S7 still started—partly for the teacher's good books.

Why it matters. CAT11 here is interest plus concurrent load, not an overloaded modelling course itself. Finishing the modelling tasks as fast as possible, so other courses could take the time, is not the same as lecturers' ECTS-scarcity story.

S8M3 — similar families confuse; paid work drains care

Case: S8. **Codes:** S8-C08, S8-C09. **Turns:** T023, T025, T027, T031.

Idea. Switching was hardest when diagrams looked alike (requirements vs class relations); an example beside the task made it fine, and knowing one family made the next easier. Models still felt like drawings next to more tangible code, so perfection mattered less than presence. A tutoring job in the master's, not competing courses, cut interest. S8 now doubts professional utility and would teach workplace benefits rather than more notation.

Why it matters. CAT14 here is lookalike-notation confusion, not S6's lost traceability. CAT11's competing load is a paid job, not ECTS stacking. The utility wish is a student interpretation, not evidence that modelling "will die."

B.4 Open-code exemplars

The following notes include the excerpts that later entered Table 4.6 (S1-C04, S1-C12, S1-C08, S2-C08, S2-C04) and one additional code per cluster used in Chapter 5.

S1-C04 — modelling useful up to a point then arbitrary

Case: S1. **Turns:** T008.

Quote. "Modelling is useful for defining how system components interact up to a certain point. After that, it can feel arbitrary and like extra documentation."

S1-C12 — symbol choices create doubt at start

Case: S1. **Turns:** T029.

Quote. "The high-level concepts were fairly straightforward, but the symbols and notation complicated things." "Is this the proper notation for this? Is this the proper arrow for that?" That made it harder to begin."

S1-C08 — tool options overwhelming for simple relations

Case: S1. **Turns:** T020, T051.

Quote. "The tool had so many options that it became completely overwhelming to know how to make a simple entity-to-box relation."

S1-C23 — learn modelling best from worked examples

Case: S1. **Turns:** T071.

Quote. "Examples are more useful for me because I learn by seeing things." "This textual description translates to this model."

S2-C08 — course difficulty from unfamiliar frameworks

Case: S2. **Turns:** T024.

Quote. "Definitely in my Software Engineering course, because I had to use frameworks with which I was unfamiliar. I had to learn them before I could start drawing."

S2-C04 — diagrams simplify concepts mind cannot hold

Case: S2. **Turns:** T014.

Quote. "Some concepts may be really complex and you cannot fit them in your mind. Making a small diagram can simplify them for you and for others who want to understand your work."

S3-C04 — earlier courses were shallow crash courses

Case: S3. **Turns:** T013.

Quote. "It was basically a crash course, not very detailed, and we did not go very deep into modelling."

Note. Industrial Management and Virtual Enterprises gave brief BPMN contact.

S4-C02 — group devalued model and abandoned it for code

Case: S4. **Turns:** T007.

Quote. "one person said it was basically unnecessary paperwork"

Note. Others pushed Spring implementation; diagram drifted from code within a week.

S4-C08 — shy student prefers private over public model sharing**Case:** S4. **Turns:** T017.*Quote.* “Not very comfortable in public. I am shy, so presenting a half-finished diagram to the whole class is stressful.”*Note.* Email or after-class showing preferred over class exposure.**S5-C04 — drafts version one without waiting for demo****Case:** S5. **Turns:** T009.*Quote.* “I rarely wait for a teacher demo.”*Note.* self-initiated draft replaces waiting for teacher example.**S6-C01 — UML class diagram lost trace from goals****Case:** S6. **Turns:** T007.*Quote.* “our pair produced a class diagram that looked syntactically correct but lost the goals”*Note.* teaching assistant confirmed classes fine but goal trace missing.**S7-C09 — grade given before feedback so no resubmit****Case:** S7. **Turns:** T023.*Quote.* “The grade was given before the feedback.”*Note.* Two-phase project; comments meant to fix the next phase or the defence, not a resubmission of the graded delivery.**S7-C16 — heavy semester made modelling something to finish fast****Case:** S7. **Turns:** T013, T035.*Quote.* “because modelling was not something that interested me much, I tried to get those course tasks done as fast as possible so I could focus more on the others.”*Note.* Second year had too many courses; modelling felt among the more boring ones. Workload itself was called adequate.**S8-C01 — bachelor project required models without teaching them****Case:** S8. **Turns:** T003.*Quote.* “I had the idea that I had to use UML, requirements engineering, and so on, because it was in the project requirements, but I had no idea how to use those notations.”*Note.* Bachelor’s barely named modelling; master’s Software Engineering was the first proper introduction.**S10-C02 — too-direct BPMN-to-UML translation of an external party****Case:** S10. **Turns:** T007, T009, T011.*Quote.* “I was trying to do a translation that was too direct from BPMN.”*Note.* Payments service clear as an external BPMN pool; UML choice among actor, class, component, or omit was the stuck point.

S10-C11 — AI as screenshot checker not model generator

Case: S10. **Turns:** T041, T043, T045, T057, T059.

Quote. "I never asked "make me the model of this whole system.""

Note. Screenshot plus short context; asked what should carry from BPMN, not a direct transform. Second opinion after self-check.

S10-C12 — split-by-notation made two versions of the system

Case: S10. **Turns:** T047, T049.

Quote. "the models started to diverge. The person doing BPMN interpreted a rule one way and the person doing UML interpreted it another way."

Note. Group invented a draft-plus-review split and a five-to-ten-minute carry-over huddle before the next model. Defence was individual.

S6-C14 — wants explicit continuous translation example

Case: S6. **Turns:** T035.

Quote. "Teach the translation path explicitly. I would like one continuous example from goals to NFR decisions to UML, with trace links documented."

Note. Also asks for integrated tools or a shared rationale table across notations.

L4, T067 — models are complementary perspectives

Case: L4. **Turns:** T067.

Quote. "Students need to understand that models are perspectives on a problem. Different models provide complementary perspectives, and together they give a better view of the problem. Making students understand this is not always easy."

Note. This sentence was not given its own open code. The codes on T067 are L4-C30, L4-C31, and L4-C32, and they quote other sentences in the same turn.

BIBLIOGRAPHY

- [1] S. Adolph, W. Hall, and P. Kruchten. “Using Grounded Theory to Study the Experience of Software Development”. In: *Empirical Software Engineering* 16.4 (2011), pp. 487–513. DOI: [10.1007/s10664-010-9152-6](https://doi.org/10.1007/s10664-010-9152-6) (cit. on pp. 12, 13, 17–19).
- [2] S. Akayama et al. “Tool Use in Software Modelling Education”. In: *Proceedings of the Educators Symposium at MODELS 2013*. Vol. 1134. CEUR Workshop Proceedings. 2013, pp. 1–22. URL: <https://ceur-ws.org/Vol-1134/paper6.pdf> (cit. on p. 10).
- [3] D. Bogdanova and M. Snoeck. “CaMeLOT: An Educational Framework for Conceptual Data Modelling”. In: *Information and Software Technology* 110 (2019), pp. 92–107. DOI: [10.1016/j.infsof.2019.02.006](https://doi.org/10.1016/j.infsof.2019.02.006) (cit. on pp. 1, 8, 42).
- [4] D. Bogdanova and M. Snoeck. “Domain Modelling in Bloom: Deciphering How We Teach It”. In: *The Practice of Enterprise Modeling (PoEM 2017)*. Vol. 305. Lecture Notes in Business Information Processing. 2017, pp. 3–17. DOI: [10.1007/978-3-319-70241-4_1](https://doi.org/10.1007/978-3-319-70241-4_1) (cit. on pp. 1, 8, 42).
- [5] D. Bork. “A Framework for Teaching Conceptual Modeling and Metamodeling Based on Bloom’s Revised Taxonomy of Educational Objectives”. In: *Proceedings of the 52nd Hawaii International Conference on System Sciences (HICSS)*. 2019, pp. 7701–7710. DOI: [10.24251/HICSS.2019.928](https://doi.org/10.24251/HICSS.2019.928) (cit. on pp. 1, 5, 8, 42).
- [6] R. A. Buchmann et al. “Conceptual Modelling in Education: A Position Paper”. In: *Joint Proceedings of the BIR 2019 Workshops and Doctoral Consortium*. Vol. 2443. CEUR Workshop Proceedings. 2019, pp. 62–67. URL: <https://ceur-ws.org/Vol-2443/paper06.pdf> (cit. on pp. 10, 44).
- [7] L. Burgueño, A. Vallecillo, and M. Gogolla. “Teaching UML and OCL Models and their Validation to Software Engineering Students: An Experience Report”. In: *Computer Science Education* 28.1 (2018), pp. 23–41. DOI: [10.1080/08993408.2018.1462000](https://doi.org/10.1080/08993408.2018.1462000) (cit. on pp. 10, 41).
- [8] J. Cámara et al. “On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML”. In: *Software and Systems Modeling* 22.3 (2023), pp. 781–793. DOI: [10.1007/s10270-023-01105-5](https://doi.org/10.1007/s10270-023-01105-5) (cit. on pp. 12, 43).

- [9] CC2020 Task Force. *Computing Curricula 2020: Paradigms for Global Computing Education*. Association for Computing Machinery and IEEE Computer Society, 2020. DOI: [10.5555/3467967.C5641384](https://doi.org/10.5555/3467967.C5641384).
- [10] S. Chakraborty and G. Liebel. “We do not understand what it says – studying student perceptions of software modelling”. In: *Empirical Software Engineering* 28.6, 149 (2023). DOI: [10.1007/s10664-023-10404-w](https://doi.org/10.1007/s10664-023-10404-w) (cit. on pp. 1, 2, 5, 9, 42).
- [11] S. Chakraborty et al. “Exploring Actions, Interactions and Challenges in Software Modelling Tasks: An Empirical Investigation with Students”. In: *Empirical Software Engineering* 31, 97 (2026). DOI: [10.1007/s10664-026-10815-5](https://doi.org/10.1007/s10664-026-10815-5) (cit. on p. 9).
- [12] K. Charmaz. *Constructing Grounded Theory: A Practical Guide through Qualitative Analysis*. London: Sage Publications, 2006 (cit. on p. 6).
- [13] I. Dey. *Grounding Grounded Theory: Guidelines for Qualitative Inquiry*. San Diego: Academic Press, 1999.
- [14] S. Foss, T. Urazova, and R. Lawrence. “Automatic Generation and Marking of UML Database Design Diagrams”. In: *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education*. Vol. 1. ACM, 2022, pp. 626–632. DOI: [10.1145/3478431.3499376](https://doi.org/10.1145/3478431.3499376) (cit. on pp. 10, 43).
- [15] G. Garaccione et al. “Gamification of business process modeling education: an experimental analysis”. In: *Software and Systems Modeling* 23.6 (2024), pp. 1569–1594. DOI: [10.1007/s10270-024-01171-3](https://doi.org/10.1007/s10270-024-01171-3) (cit. on pp. 11, 46).
- [16] B. G. Glaser and A. L. Strauss. *The Discovery of Grounded Theory: Strategies for Qualitative Research*. Chicago, IL: Aldine Publishing Company, 1967 (cit. on p. 6).
- [17] T. Gorschek, E. Tempero, and L. Angelis. “On the Use of Software Design Models in Software Development Practice: An Empirical Investigation”. In: *Journal of Systems and Software* 95 (2014), pp. 176–193. DOI: [10.1016/j.jss.2014.03.082](https://doi.org/10.1016/j.jss.2014.03.082) (cit. on pp. 1, 5).
- [18] I. Graßl et al. “Domain Diversity, Motivation, Inclusion, and Feedback in Software Modelling Education”. In: *FSE Companion '26: Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. ACM, 2026, pp. 1051–1062. DOI: [10.1145/3803437.3805791](https://doi.org/10.1145/3803437.3805791) (cit. on pp. 1, 2, 11, 44).
- [19] I. Hammouda et al. “CASE Tools versus Pencil and Paper: A Student’s Perspective on Modeling Software Design”. In: *Proceedings of the Educators Symposium at MODELS 2014*. Vol. 1346. CEUR Workshop Proceedings. 2014. URL: https://ceur-ws.org/Vol-1346/edusymp2014_paper_4.pdf (cit. on p. 10).
- [20] R. W. Hasker and M. Rowe. “UMLint: Identifying Defects in UML Diagrams”. In: *2011 ASEE Annual Conference & Exposition*. Vancouver, BC: ASEE Conferences, 2011, pp. 22.1558.1–22.1558.14. DOI: [10.18260/1-2--18929](https://doi.org/10.18260/1-2--18929) (cit. on pp. 10, 43).

- [21] R. Hoda. “Socio-Technical Grounded Theory for Software Engineering”. In: *IEEE Transactions on Software Engineering* 48.10 (2022), pp. 3808–3832. DOI: [10.1109/TSE.2021.3106280](https://doi.org/10.1109/TSE.2021.3106280) (cit. on pp. 3, 7, 12, 13).
- [22] R. Holmes, M. Allen, and M. Craig. “Dimensions of Experientialism for Software Engineering Education”. In: *Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training Track (ICSE-SEET '18)*. New York, NY, USA: ACM, 2018, pp. 31–39. DOI: [10.1145/3183377.3183380](https://doi.org/10.1145/3183377.3183380) (cit. on p. 11).
- [23] J. Kienzle et al. “Requirements for modelling tools for teaching”. In: *Software and Systems Modeling* 23.5 (2024), pp. 1055–1073. DOI: [10.1007/s10270-024-01192-y](https://doi.org/10.1007/s10270-024-01192-y) (cit. on p. 10).
- [24] D. R. Krathwohl. “A Revision of Bloom’s Taxonomy: An Overview”. In: *Theory Into Practice* 41.4 (2002), pp. 212–218. DOI: [10.1207/s15430421tip4104_2](https://doi.org/10.1207/s15430421tip4104_2) (cit. on pp. 3, 6, 8, 16, 21).
- [25] T. Kühne. “What is a Model?” In: *Language Engineering for Model-Driven Software Development*. Ed. by J. Bézivin and R. Heckel. Vol. 4101. Dagstuhl Seminar Proceedings (DagSemProc). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2005, pp. 1–10. DOI: [10.4230/DagSemProc.04101.15](https://doi.org/10.4230/DagSemProc.04101.15) (cit. on pp. 1, 5).
- [26] T. Li et al. “Understanding the Challenges and Requirements for Facilitating iStar Learning: An Empirical Study with iStar Learners”. In: *Information and Software Technology* 187 (2025), p. 107839. DOI: [10.1016/j.infsof.2025.107839](https://doi.org/10.1016/j.infsof.2025.107839) (cit. on pp. 1, 3, 9, 13, 17, 42).
- [27] G. Liebel, O. Badreddin, and R. Heldal. “Model Driven Software Engineering in Education: A Multi-Case Study on Perception of Tools and UML”. In: *2017 IEEE 30th Conference on Software Engineering Education and Training (CSEET)*. 2017, pp. 124–133. DOI: [10.1109/CSEET.2017.29](https://doi.org/10.1109/CSEET.2017.29) (cit. on pp. 1, 2, 9, 43, 44, 46).
- [28] G. Liebel et al. “Human factors in model-driven engineering: future research goals and initiatives for MDE”. In: *Software and Systems Modeling* 23 (2024), pp. 801–819. DOI: [10.1007/s10270-024-01188-8](https://doi.org/10.1007/s10270-024-01188-8) (cit. on p. 10).
- [29] J. M. Lourenço. *The NOVAtthesis L^AT_EX Template User’s Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomlourenco/novathesis/raw/main/template.pdf>.
- [30] I. Maslov. “Towards Empirically Validated Process Modelling Education Using a BPMN Formalism”. In: *Research Challenges in Information Science (RCIS 2022)*. Vol. 446. Lecture Notes in Business Information Processing. Springer, 2022, pp. 803–810. DOI: [10.1007/978-3-031-05760-1_58](https://doi.org/10.1007/978-3-031-05760-1_58) (cit. on pp. 1, 9).

- [31] I. Maslov, S. Poelmans, and K. Rosenthal. “Conceptual Modeling Education: A Bibliometric Literature Review and Avenues for Future Research”. In: *Business & Information Systems Engineering* (2025), pp. 1–33. DOI: [10.1007/s12599-025-00930-w](https://doi.org/10.1007/s12599-025-00930-w) (cit. on pp. 1, 2, 8, 42, 45).
- [32] D. Moody. “The “Physics” of Notations: Toward a Scientific Basis for Constructing Visual Notations in Software Engineering”. In: *IEEE Transactions on Software Engineering* 35.6 (2009), pp. 756–779. DOI: [10.1109/TSE.2009.67](https://doi.org/10.1109/TSE.2009.67) (cit. on pp. 9, 41).
- [33] Object Management Group. *OMG Unified Modeling Language (UML), Version 2.5.1*. Formal specification, December 2017. 2017. URL: <https://www.omg.org/spec/UML/2.5.1/PDF> (cit. on pp. 9, 41).
- [34] M. Petre. “UML in Practice”. In: *2013 35th International Conference on Software Engineering (ICSE)*. 2013, pp. 722–731. DOI: [10.1109/ICSE.2013.6606618](https://doi.org/10.1109/ICSE.2013.6606618) (cit. on pp. 1, 5).
- [35] Y. Sedelmaier and D. Landes. “A Research Agenda for Identifying and Developing Required Competencies in Software Engineering”. In: *International Journal of Engineering Pedagogy (iJEP)* 3.2 (2013), pp. 30–35. DOI: [10.3991/ijep.v3i2.2448](https://doi.org/10.3991/ijep.v3i2.2448) (cit. on p. 8).
- [36] K. Sikkel and M. Daneva. “Teaching consistency of UML specifications”. In: *2010 5th International Workshop on Requirements Engineering Education and Training*. IEEE, 2010, pp. 17–19. DOI: [10.1109/REET.2010.5633113](https://doi.org/10.1109/REET.2010.5633113) (cit. on pp. 11, 44).
- [37] W. Silva et al. “What are the differences between group and individual modeling when learning UML?” In: *Proceedings of the XXXII Brazilian Symposium on Software Engineering*. ACM, 2018, pp. 308–317. DOI: [10.1145/3266237.3266255](https://doi.org/10.1145/3266237.3266255) (cit. on pp. 12, 44).
- [38] D. Sims and F. Cilliers. “Qualitatively Speaking: Deciding How Much Data and Analysis Is Enough”. In: *African Journal of Health Professions Education* 15.1 (2023), pp. 2–3. DOI: [10.7196/AJHPE.2023.v15i1.1657](https://doi.org/10.7196/AJHPE.2023.v15i1.1657).
- [39] K.-J. Stol, P. Ralph, and B. Fitzgerald. “Grounded Theory in Software Engineering Research: A Critical Review and Guidelines”. In: *Proceedings of the 38th International Conference on Software Engineering*. 2016, pp. 120–131. DOI: [10.1145/2884781.2884833](https://doi.org/10.1145/2884781.2884833) (cit. on pp. 3, 6, 12, 13, 17–19).
- [40] A. Strauss and J. Corbin. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. 2nd ed. Thousand Oaks, CA: Sage Publications, 1998 (cit. on p. 6).
- [41] P. Vemuri et al. “Using Formative Assessment and Feedback to Train Novice Modelers in Business Process Modeling”. In: *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. 2023, pp. 130–137. DOI: [10.1109/MODELS-C59198.2023.00037](https://doi.org/10.1109/MODELS-C59198.2023.00037) (cit. on pp. 10, 43).

- [42] C. Verbruggen and M. Snoeck. “Exploratory Study on Students’ Understanding of Multi-Perspective Modelling”. In: *Enterprise, Business-Process and Information Systems Modeling*. Ed. by A. Augusto et al. Vol. 450. Lecture Notes in Business Information Processing. Springer, 2022, pp. 321–335. DOI: [10.1007/978-3-031-07475-2_22](https://doi.org/10.1007/978-3-031-07475-2_22) (cit. on pp. 11, 44).
- [43] J. Whittle et al. “Teaching in a Software Design Studio: Implications for Modeling Education”. In: *Proceedings of the Educators Symposium at MODELS 2014*. Vol. 1346. CEUR Workshop Proceedings. 2014. URL: https://ceur-ws.org/Vol-1346/edusymp2014_paper_1.pdf (cit. on p. 11).
- [44] S. Zschaler et al. “What do we need from modeling tools for teaching? A survey of the community of higher-education modeling teachers”. In: *Software and Systems Modeling* (2025). DOI: [10.1007/s10270-025-01336-8](https://doi.org/10.1007/s10270-025-01336-8) (cit. on p. 10).

